Table of Contents

1. Introduction 11
2. Data Preprocessing..... 12
2.1 Introduction 12
2.2 Dataset Overview 12
2.3 Data Quality Assessment 14
2.3.1 Missing Values..... 14
2.3.2 Duplicate Records 15
2.4 Outlier Detection and Removal 15
2.4.1 Why Outliers Matter 15
2.4.2 Visual Inspection with Box Plots 15
2.4.3 IQR-Based Outlier Detection 17
2.4.4 Outlier Removal 18
2.4.5 Post-Removal Validation 19
2.5 Categorical Variable Encoding 19
2.5.1 Why Encoding Is Necessary 19
2.5.2 One-Hot Encoding 20
2.6 Feature Standardisation 21
2.6.1 Why Standardise? 21
2.6.2 StandardScaler 21
3 Feature Exploration 24
3.1. Introduction 24
3.2. Summary Statistics..... 24
3.3. Univariate Analysis 25
3.3.1 Target Variable Heart Disease Presence 25
3.3.2 Categorical Features..... 27
3.3.3 Numerical Features 35
3.4. Bivariate Analysis 40
3.4.1 Categorical Features vs Target..... 40
3.4.2 Histograms for Numerical Features by Target 45
3.4.3 Pair Plot 50

3.5. Correlation Heatmap 51
4 Traditional Machine Learning Models 54
4.1. Introduction 54
4.2. Data Splitting 54
4.3. Evaluation Metrics 55
4.4. Baseline Models 56
4.4.1 Model 01 - Logistic Regression 56
4.4.2 Model 02 - Random Forest Classifier 58
4.4.3 Model 03 - Support Vector Machine (SVM) 60
4.4.4 Model 04 - Decision Tree 62
4.4.5 Model 05 - AdaBoost Classifier 64
4.4.6 Model 06 - XGBoost Classifier 66
4.4.7 Baseline Model Comparison Summary 68
4.5. Hyperparameter Tuning 69
4.5.1 Model 01 - Tuned Logistic Regression 69
4.5.2 Model 02 - Tuned Random Forest 71
4.5.3 Model 03 - Tuned Support Vector Machine 73
4.5.4 Model 04 - Tuned Decision Tree 75
4.5.5 Model 05 - Tuned AdaBoost 77
4.5.6 Model 05 - Tuned XGBoost 79
4.6. Tuned Model Comparison Summary 81
4.7. Cross-Validation Scores 82
4.8. Overfitting and Underfitting Analysis 82
4.9. Baseline vs. Tuned Comparison 84
5. Visualizations 85
5.1 Feature Importance 85
5.2 ROC Curve Comparison 86
5.3 Overall Comparison of Model Performance 87
6. Overall Results, Insights and Challenges 88
6.1 Overall Results 88
6.2 Key Insights 89

6.3 Challenges and Limitations..... 90
7. Conclusion 90
References .....
92

Table of Figures

Figure 1Loading the dataset 12
Figure 2 Dataset structure output from df.info() 13
Figure 3 Missing value check 14
Figure 4 Duplicate row check 15
Figure 5 Selecting numeric columns..... 15
Figure 6 Box plot loop 15
7 Box plot for each column 16
Figure 8 IQR lower and upper bound formula 17
Figure 9 detect_outliers_iqr() function definition..... 17
Figure 10 Collecting outlier indices and removing rows 18
Figure 11 One-Hot Encoding using pd.get_dummies() 20
Figure 12 Applying StandardScaler to numerical features 22
Figure 13 Preprocessed df_encoded DataFrame output 22
Figure 14 Summary statistics output from df_encoded.describe() 24
15 Summary statistics 24
16 Proportion of Heart Disease Presence Results 26
17 Proportion of Sex in the Dataset Results 27
18 Proportion of Chest Pain Types Results 28
19 Proportion of Fasting Blood Sugar Results 29
20 Proportion of Resting Electrocardiographic Results..... 30
21 Proportion of Exercise-Induced Angina Results..... 31
22 Proportion of slope of peak exercise 32
23 Proportion of Number of major vessels 33
24 Proportion of Thalassemia Types 34
25 Distribution of age 35
26 Boxplot of cholesterol levels (chol) 36
27 Distribution of Blood Pressure.....

37 28 Boxplot of Thalach (Heart Rate)..... 38 29 Distribution of Oldpeak 39
30 Distribution of heart disease by gender 40
31 Distribution of heart disease by chest pain type 41
32 Distribution of heart disease by fasting blood sugar (fbs) 42
33 Distribution of heart disease by exercise-induced angina (exang) 43
34 Distribution of heart disease by resting ECG (restecg) 44
35 Distribution of age (numerical feature)..... 45
36 Distribution of Cholesterol(numerical feature) 46
37 Distribution of Trestbps (numerical feature) 47
38 Distribution of Thalach (numerical feature) 48
39 Distribution of Oldpeak (numerical feature) 49
40 Histograms and Pair Plot 51
41 Correlation Heatmap 52
Figure 42 Column selection list for pair plot analysis ..... 54 Figure 43evaluate_model() helper function definition 55
Figure 44 Baseline Logistic Regression training code 56
Figure 45 Logistic Regression baseline evaluation metrics output 56
46 Confusion matrix of Logistic Regression model 57
47 ROC Curve - Logistic Regression 57
Figure 48 Baseline Random Forest training code ..... 58 Figure 49 Random Forest baseline evaluation metrics output 58
50 Confusion matrix of Random Forest model..... 59
51 ROC Curve - Random Forest 59
Figure 52 Baseline SVM training code .....

60 Figure 53 SVM baseline evaluation metrics output 60
54 Confusion matrix of SVM model 61
55 ROC Curve - SVM..... 61
Figure 56 Baseline Decision Tree training code 62
Figure 57 Decision Tree baseline evaluation metrics output 62
58 Confusion matrix of Decision Tree model..... 63
59 ROC Curve- Decision Tree 63
Figure 60 Baseline AdaBoost training code 64
Figure 61 AdaBoost baseline evaluation metrics output 64
62 Confusion matrix of AdaBoost model 65
63 ROC Curve- AdaBoost 65
Figure 64 Baseline XGBoost training code 66
Figure 65 XGBoost baseline evaluation metrics output 66
66 Confusion matrix of XGBoost model 67
67 ROC curve of tuned XGBoost model 68
Figure 68 All six baseline models comparison summary output 68
Figure 69 Logistic Regression hyperparameter grid definition 69
Figure 70 Tuned Logistic Regression best parameters and metrics output 70
71 Confusion matrix of tuned Logistic Regression model 70
72ROC curve of tuned Logistic Regression model 71
Figure 73 Random Forest hyperparameter grid definition 71
Figure 74 Tuned Random Forest best parameters and metrics output..... 72
75 Confusion matrix of tuned Random Forest model 72
76ROC curve of tuned Random Forest model 73
Figure 77 SVM hyperparameter grid definition 73
Figure 78 Tuned SVM best parameters and metrics output 74
79 Confusion matrix of tuned SVM model..... 74

80 ROC curve of tuned Support Vector Machine model 75
Figure 81 Decision Tree hyperparameter grid definition .....
75 Figure 82 Tuned Decision Tree best parameters and metrics output
..... 76
83 Confusion matrix of tuned Decision Tree model 76
84 ROC curve of tuned Decision Tree model 77
Figure 85 AdaBoost hyperparameter grid definition .....
77 Figure 86 Tuned AdaBoost best parameters and metrics output
..... 78
87 Confusion matrix of tuned AdaBoost model 78
88 ROC curve of tuned AdaBoost model 79
Figure 89 XGBoost hyperparameter grid definition .....
79 Figure 90 XG Boost best parameters and metrics output
..... 80 91 Confusion matrix of tuned XGBoost model
..... 80 92 ROC curve of tuned XGBoost model
..... 81 Figure 93 Tunned Model Comparison
summary ..... 81 94Cross-Validation Scores for Best
Tuned Models..... 82 Figure 95 Overfitting and
underfitting analysis 83
96 Overfitting/Underfitting Summary 83
97 Baseline vs Tuned Final Comparison 84
98 Feature Importance for Tree-Based Models 85
99 Comparison of Model Performance 87

Table of Tables

Table 1: Dataset Features 14
Table 2 Summarise Results 18
Table 3 updated bounds & results 19
Table 4 Categorical Features 20
Table 5 Summary of preprocessing 23

1. Introduction

This study focuses on the application of machine learning techniques for the prediction of heart disease using the Cleveland Heart Disease dataset. The project follows a structured pipeline beginning with rigorous data preprocessing, followed by exploratory data analysis (EDA), deployment of numerous classical machine learning models, and ultimately comparative evaluation of their performance.

The preparation stage ensured that the dataset was clean, consistent, and acceptable for modelling by addressing concerns such as outliers, categorical encoding, and feature scaling. Exploratory analysis provided insights into feature distributions and relationships, identifying key predictors such as chest pain type, maximum heart rate, ST depression, and number of vessels.

Six machine learning models were implemented, including Logistic Regression, Random Forest, Support Vector Machine (SVM), Decision Tree, AdaBoost, and XGBoost. Each model was tested using numerous performance indicators, and then enhanced through hyperparameter tuning to promote generality and predictive capability.

The ultimate purpose of the study was to discover the most suited model for accurate and reliable heart disease prediction while also evaluating the strengths, weaknesses, and behaviour of different algorithms in a medical categorization setting.

2. Data Preprocessing

2.1 Introduction

The first stage of any machine learning project is to prepare the data before any model is trained. This process, called data preprocessing, ensures that the dataset is clean, consistent, and in the right format for algorithms to learn from. This section documents all the preprocessing steps applied to the Cleveland Heart Disease Dataset, including loading the data, checking for data quality issues, removing anomalies, encoding categorical variables, and standardising numerical features.

2.2 Dataset Overview

The Cleveland Heart Disease Dataset is a widely used benchmark dataset in medical machine learning research. It was originally collected at the Cleveland Clinic Foundation and contributed to the UCI Machine Learning Repository (Detrano, 1989) .The version used here is a pre-processed

binary classification variant containing 303 patient records and 14 columns 13 clinical and demographic input features plus one binary target variable indicating the presence or absence of heart disease (0 = No Disease, 1 = Disease).

The dataset was loaded in a Google Colab environment using the pandas library (McKinney, 2010), with the CSV file read into a DataFrame as follows:

```
df=pd.read_csv(io.BytesIO(data['Heart_disease_cleveland_new (1).csv']), encoding='latin1')
df.head()
```

Figure 1Loading the dataset

The initial inspection confirmed the dataset dimensions using df.shape, which returned (303, 14), and df.info () which showed that 13 columns are of integer type and 1 column (oldpeak) is of float type. All 14 columns reported 303 non-null entries, giving a complete dataset with no structural gaps.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   ï»¿age    303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64
 11  ca        303 non-null    int64
 12  thal      303 non-null    int64
 13  target    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

Figure 2 Dataset structure output from df.info()

Table 1: Dataset Features

No.	Feature	Description	Type
1	age	Age of the patient in years	Numerical
2	sex	Sex (1 = Male, 0 = Female)	Categorical
3	cp	Chest pain type (0 to 3)	Categorical
4	trestbps	Resting blood pressure in mm Hg	Numerical
5	chol	Serum cholesterol in mg/dl	Numerical
6	fbs	Fasting blood sugar greater than 120 mg/dl	Categorical
7	restecg	Resting ECG results (0, 1, 2)	Categorical
8	thalach	Maximum heart rate achieved	Numerical
9	exang	Exercise-induced angina (1 = Yes, 0 = No)	Categorical
10	oldpeak	ST depression induced by exercise	Numerical
11	slope	Slope of peak exercise ST segment	Categorical
12	ca	Number of major vessels (0 to 4)	Categorical
13	thal	Thalassemia type (0, 1, 2, 3)	Categorical
14	target	Heart disease present (1 = Yes, 0 = No)	Target Variable

Table 1: Dataset Features

2.3 Data Quality Assessment

2.3.1 Missing Values

One of the most common problems in real-world datasets is missing data, which can cause machine learning models to behave unpredictably or produce biased results (García, 2015). To check this, the following command was used:

```
df.isnull().sum()
```

	0
ï»¿age	0
sex	0
cp	0
trestbps	0
chol	0
fbs	0
restecg	0
thalach	0
exang	0
oldpeak	0
slope	0
ca	0
thal	0
target	0

dtype: int64

Figure 3 Missing value check

The output showed a count of zero (0) for every column, meaning no missing values were found across the entire dataset. This is because the version of the Cleveland dataset used here is already pre-processed any records that originally contained missing values (marked as '?' in the UCI repository) had been addressed prior to this analysis. As a result, no imputation or row removal for missing data was required.

2.3.2 Duplicate Records

Duplicate rows can introduce bias by making the model learn the same patterns multiple times. The following check was performed:

```
df.duplicated().sum()
```

```
np.int64(0)
```

Figure 4 Duplicate row check

The output returned 0, confirming that all 303 records are unique. No rows needed to be dropped at this stage.

2.4 Outlier Detection and Removal

2.4.1 Why Outliers Matter

Outliers are data points that fall unusually far from the rest of the distribution. In a machine learning context, extreme values can distort the model's decision boundaries, inflate the importance of certain features, and reduce overall prediction accuracy (Aggarwal, 2017). This is especially critical in medical datasets where extreme readings may represent measurement errors rather than genuine patient conditions.

2.4.2 Visual Inspection with Box Plots

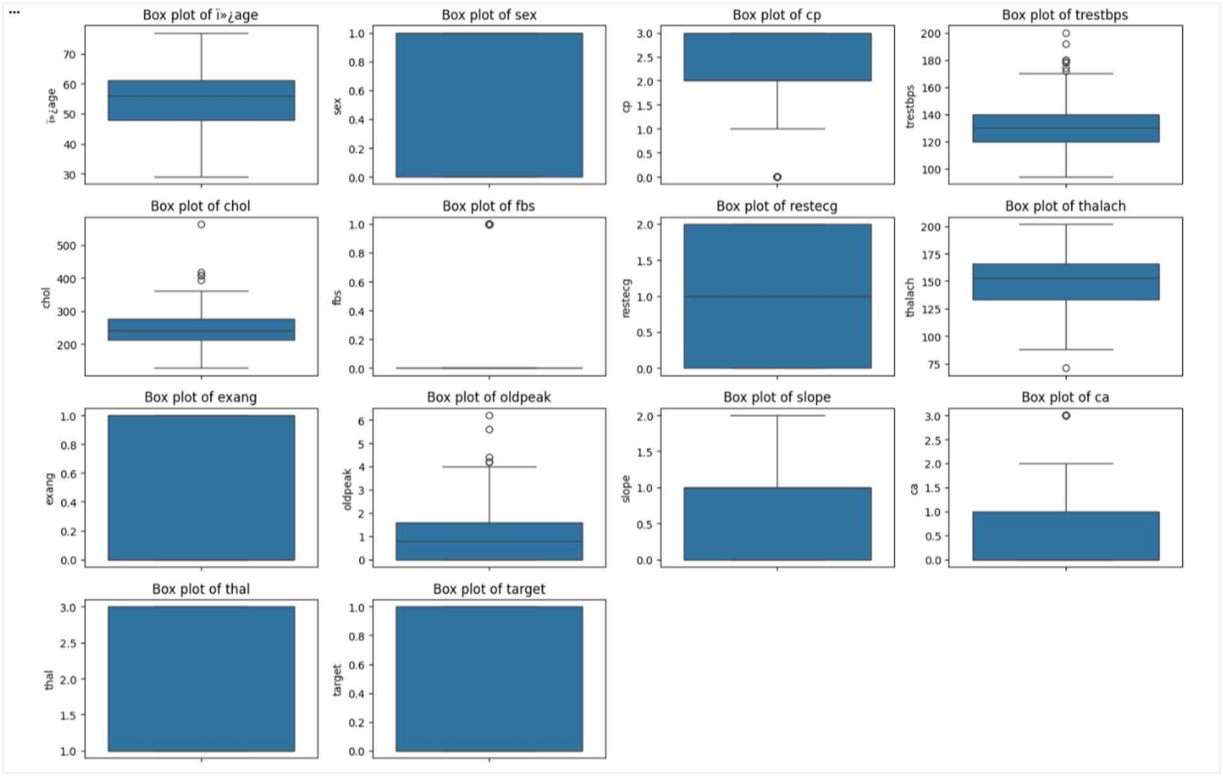
The first step in outlier detection was visual inspection. Box plots were generated for all numerical columns using seaborn and matplotlib (Hunter, 2007).The grid of box plots used the following approach:

```
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
```

Figure 5 Selecting numeric columns

```
for i, col in enumerate(numeric_cols):
    plt.subplot(4, 4, i + 1)
    sns.boxplot(y=df[col])
    plt.title(f'Box plot of {col}')
```

Figure 6 Box plot loop



7 Box plot for each column

In a box plot, the box represents the middle 50% of the data (from Q1 to Q3). Points that fall beyond the whiskers (which extend 1.5 times the IQR from each quartile) are flagged as potential outliers. The visual inspection revealed the following:

- trestbps (Resting Blood Pressure): Several values appeared above the upper whisker, indicating unusually high blood pressure readings.
- chol (Serum Cholesterol): A notable number of high-value outliers were visible, consistent with clinically extreme cholesterol levels.
- thalach (Maximum Heart Rate): One or more values appeared below the lower whisker, suggesting unusually low maximum heart rate readings.
- oldpeak (ST Depression): The distribution appeared right-skewed, with high values extending well beyond the upper whisker.

The remaining features (age, sex, fbs, restecg, exang, slope, ca, thal, target) did not show obvious outliers through visual inspection and were not included in the formal outlier removal step.

2.4.3 IQR-Based Outlier Detection

Following the visual inspection, a formal quantitative approach was applied: the Interquartile Range (IQR) method. This method was chosen because it is robust to extreme values and makes no assumption about the shape of the data distribution, making it well-suited for medical datasets (Tukey, 1977)

The IQR is the range between the 25th percentile (Q1) and the 75th percentile (Q3). Any value outside the following bounds is classified as an outlier:

```
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

Figure 8 IQR lower and upper bound formula

```
IQR = Q3 - Q1
```

A custom function was defined to apply this formula to each selected numerical column and return the outlier rows along with the computed bounds:

```
def detect_outliers_iqr(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Get rows that are outliers
    outliers = df[(df[column] < lower_bound) | (df[column] > upper_bound)]

    return outliers, lower_bound, upper_bound
```

Figure 9 detect_outliers_iqr() function definition

This function was applied to the four numerical features identified during visual inspection.

Table 2 summarises the results:

Feature	Lower Bound	Upper Bound	Outliers Detected
trestbps	94.00	166.00	9
chol	126.50	360.50	5
thalach	84.50	202.50	1
oldpeak	-2.40	3.60	5

Table 2 Summarise Results

A total of 20 individual outlier instances were detected across the four features. However, because some rows may contain outliers in more than one feature simultaneously, the actual number of rows removed was determined by taking the union of all outlier row indices to avoid doublecounting.

2.4.4 Outlier Removal

All unique outlier row indices collected across the four features were combined using Python's pd.Index.union() method, and the corresponding rows were dropped from the original DataFrame to produce the cleaned dataset df_cleaned:

```
# Collect all outlier indices
all_outlier_indices = pd.Index([])
for col in outlier_columns:
    indices = detect_outliers_iqr(df, col)
    all_outlier_indices = all_outlier_indices.union(indices)

print(f"Total number of rows before outlier removal: {len(df)}")
print(f"Number of outlier rows to be removed: {len(all_outlier_indices)}")

# Remove the outlier rows from the DataFrame
df_cleaned = df.drop(all_outlier_indices)
```

Figure 10 Collecting outlier indices and removing rows

Row deletion was chosen over imputation because in a clinical dataset, extreme physiological values (such as abnormally high cholesterol) are more likely to represent data entry errors or

measurement anomalies than genuine patient conditions. Retaining such values could introduce noise and bias the model during training. Since the outlier rows represent a small fraction of the 303 records, deletion is a safe strategy that preserves the integrity of the remaining data (Aggarwal, 2017).

2.4.5 Post-Removal Validation

After deletion, the IQR detection function was re-applied to df_cleaned to confirm that no outlier values remained.

Table 3 shows the updated bounds and results:

Feature	Lower Bound	Upper Bound	Outliers Remaining
trestbps	90.00	170.00	None
chol	120.38	361.38	None
thalach	82.38	219.38	None
oldpeak	-2.40	4.00	None

Table 3 updated bounds & results

The absence of outliers in all four features confirms that the cleaning process was successful. The bounds shifted slightly from Table 2 because the IQR is recalculated on the smaller cleaned dataset once extreme values are removed, the quartiles shift inward, tightening the acceptable range. The fact that no new outliers were flagged confirms the cleaning was complete.

2.5 Categorical Variable Encoding

2.5.1 Why Encoding Is Necessary

Although several features in the dataset are stored as integers, they do not represent continuous numerical quantities they represent categories. For example, the feature 'cp' (chest pain type) uses values 0, 1, 2, and 3 to label four distinct categories. If these integers were passed directly to a machine learning model, the model would incorrectly interpret them as having a meaningful numerical order (i.e., type 3 is 'three times more' than type 1), which carries no clinical meaning (Géron, 2022)

Eight features were identified as requiring encoding:

Feature	Unique Values	Nature
sex	0, 1	Binary (Female, Male)
cp	0, 1, 2, 3	Nominal (Chest pain type)
fbs	0, 1	Binary (Blood sugar threshold)
restecg	0, 1, 2	Nominal (ECG result category)
exang	0, 1	Binary (Exercise-induced angina)
slope	0, 1, 2	Ordinal (ST slope direction)
ca	0, 1, 2, 3, 4	Ordinal (Number of major vessels)
thal	0, 1, 2, 3	Nominal (Thalassemia type)

Table 4 Categorical Features

2.5.2 One-Hot Encoding

One-Hot Encoding (OHE) was applied to all eight categorical features using pandas get_dummies() with drop_first=False. OHE converts each category into its own binary (0 or 1) indicator column. This means the model treats each category as an independent dimension rather than assuming any order or magnitude between them (Pedregosa, 2011).

```
categorical_cols = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal']

# Apply one-hot encoding using pandas get_dummies
# This creates new columns for each category and drops the original categorical columns
df_encoded = pd.get_dummies(df_cleaned, columns=categorical_cols, drop_first=False)
```

Figure 11 One-Hot Encoding using pd.get_dummies()

The encoding expanded the number of columns from 14 to 31. This increase reflects the creation of binary indicator columns for each unique category: sex contributes 2 columns, cp contributes 4, fbs contributes 2, restecg contributes 3, exang contributes 2, slope contributes 3, ca contributes 5, and thal contributes 4 totalling 25 new binary columns replacing the original 8. The 5 numerical columns and the target variable remain unchanged.

2.6 Feature Standardisation

2.6.1 Why Standardise?

The five numerical features (age, trestbps, chol, thalach, oldpeak) are measured on very different scales. For example, serum cholesterol values typically range from 120 to 360 mg/dl, while age ranges from 29 to 77. Many machine learning algorithms particularly those based on distance calculations or gradient descent are sensitive to these scale differences and will give disproportionate weight to features with larger numeric ranges (Géron, 2022). Standardisation ensures all features contribute equally.

2.6.2 StandardScaler

StandardScaler from scikit-learn (Pedregosa, 2011)was used to transform each numerical feature so that it has a mean of 0 and a standard deviation of 1. The formula applied to each value is:

= (x - mean) / standard_deviation

Where x is the original value, mean is the column average, and standard deviation measures how spread out the values are. After transformation, each numerical feature is centred around 0 with unit variance. The scaler was applied only to the 5 numerical columns (age, trestbps, chol, thalach, oldpeak), leaving the binary one-hot encoded columns and the target variable unchanged.

```
from sklearn.preprocessing import StandardScaler

# Get columns that are not boolean (created by get_dummies) and not the 'target' column
features_to_scale = df_encoded.select_dtypes(include=['int64', 'float64']).columns.tolist()
if 'target' in features_to_scale:
    features_to_scale.remove('target')

print(f"Features to be scaled: {features_to_scale}")

# Initialize the StandardScaler
scaler = StandardScaler()

# Apply the scaler to the selected features
df_encoded[features_to_scale] = scaler.fit_transform(df_encoded[features_to_scale])
```

Figure 12 Applying StandardScaler to numerical features

The resulting df_encoded DataFrame contains all 31 columns in their final preprocessed form, ready for use in model training and evaluation.

DataFrame after standardizing numerical features:

	ï»¿age	trestbps	chol	thalach	oldpeak	target	sex_0	sex_1	cp_0	cp_1	...	slope_0	slope_1	slope_2	ca_0	ca_1	ca_2	ca_3	thal_1	thal_2	thal_3
0	0.980969	0.973671	-0.213548	0.000000	1.319305	0	False	True	True	False	...	False	False	True	True	False	False	False	False	True	False
1	1.420393	1.946200	0.973426	-1.856679	0.538310	1	False	True	False	False	...	False	True	False	False	False	False	True	True	False	False
2	1.420393	-0.647211	-0.303131	-0.928339	1.612179	1	False	True	False	False	...	False	True	False	False	False	True	False	False	False	True
3	-1.875291	0.001141	0.167180	1.635646	2.490799	0	False	True	False	False	...	False	False	True	True	False	False	False	True	False	False
4	-1.435866	0.001141	-0.863025	0.972546	0.440685	0	True	False	False	True	...	True	False	False	True	False	False	False	True	False	False

Figure 13 Preprocessed df_encoded DataFrame output

Table 5: Summary of Preprocessing Steps

Step	Action	Outcome
Data Loading	Loaded CSV using pandas	303 rows × 14 columns
Missing Values	Checked with isnull().sum()	0 missing values found
Duplicates	Checked with duplicated().sum()	0 duplicate rows found
Outlier Detection	IQR method on 4 numerical features	Outliers flagged in trestbps, chol, thalach, oldpeak
Outlier Removal	Dropped rows via index union	df_cleaned with no outlier rows
Categorical Encoding	One-Hot Encoding (get_dummies)	Columns expanded from 14 to 31
Standardisation	StandardScaler (mean=0, std=1)	5 numerical features scaled

Table 5 Summary of preprocessing

3 Feature Exploration

3.1. Introduction

Feature exploration, often called Exploratory Data Analysis (EDA), is the process of understanding what the data looks like before any model is built. The goal is to uncover patterns, relationships, and the overall behaviour of each variable both individually and in relation to the target (heart disease presence). This step is critical because it guides feature selection and helps identify which variables are likely to be the most informative for prediction (Tukey, 1977).

This section covers three main analyses: summary statistics (a numerical overview of each feature), univariate analysis (examining each feature independently), and bivariate analysis (comparing features against each other and against the target variable), concluding with a correlation heatmap.

3.2 Summary Statistics

Summary statistics were generated on the fully preprocessed and encoded DataFrame (df_encoded) using the describe() function, which outputs the count, mean, standard deviation, minimum, maximum, and quartile values for every column.

	i»¿age	trestbps	chol	thalach	oldpeak	target
count	2.840000e+02	2.840000e+02	2.840000e+02	2.840000e+02	2.840000e+02	284.000000
mean	3.408854e-16	-6.098408e-17	5.003822e-17	1.250956e-17	6.254778e-17	0.443662
std	1.001765e+00	1.001765e+00	1.001765e+00	1.001765e+00	1.001765e+00	0.497693
min	-2.754140e+00	-2.332929e+00	-2.609893e+00	-2.740812e+00	-9.260576e-01	0.000000
25%	-7.767292e-01	-6.472114e-01	-7.118535e-01	-7.183579e-01	-9.260576e-01	0.000000
50%	1.021198e-01	1.141466e-03	-6.797582e-02	1.326199e-01	-3.403107e-01	0.000000
75%	6.514004e-01	6.494943e-01	6.374902e-01	7.957195e-01	6.359341e-01	1.000000
max	2.518955e+00	2.594553e+00	2.630712e+00	2.298745e+00	2.976922e+00	1.000000

Figure 14 Summary statistics output from df_encoded.describe()

15 Summary statistics

The key observations from the summary statistics are as follows:

- The five numerical features (age, trestbps, chol, thalach, oldpeak) have been successfully standardised. After StandardScaler is applied, each of these features shows a mean of approximately 0.0 and a standard deviation of approximately 1.0. This confirms that the standardisation step worked correctly and all numerical features are now on a comparable scale.

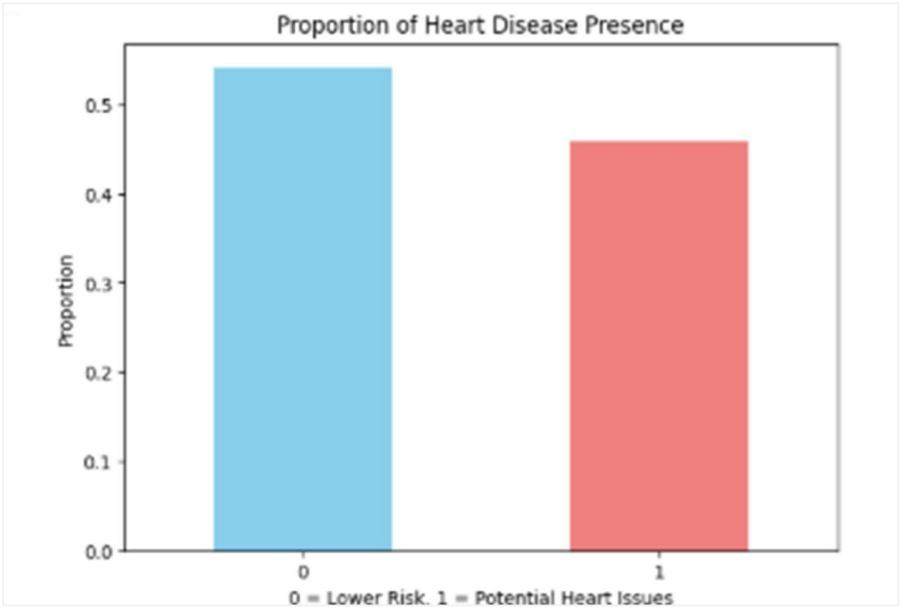
- All one-hot encoded columns (created from sex, cp, fbs, restecg, exang, slope, ca, and thal) contain only binary values of 0 and 1. Their means represent the proportion of records belonging to each category. For example, a mean of 0.68 for sex_1 would indicate that approximately 68% of patients are male.
- The target column contains values of 0 (No Disease) and 1 (Disease). A mean slightly below 0.5 indicates that the dataset is roughly though not perfectly balanced between the two classes, which is a favourable condition for training classification models without needing to apply resampling techniques.
- The minimum and maximum values for standardised numerical features are symmetrically distributed around 0, with the range of values reflecting the original spread of the data before scaling. No unexpected extreme values appear in the post-scaling statistics, confirming the effectiveness of the preprocessing pipeline.

3.3 Univariate Analysis

Univariate analysis examines each feature in isolation to understand its distribution, frequency, and behaviour before considering its relationship with other variables. The analysis is split between categorical (qualitative) and numerical (quantitative) features.

3.3.1 Target Variable Heart Disease Presence

The target variable indicates whether a patient has heart disease (1) or not (0). The value counts were computed using `df['target'].value_counts(normalize=True)` and visualised as a bar chart.



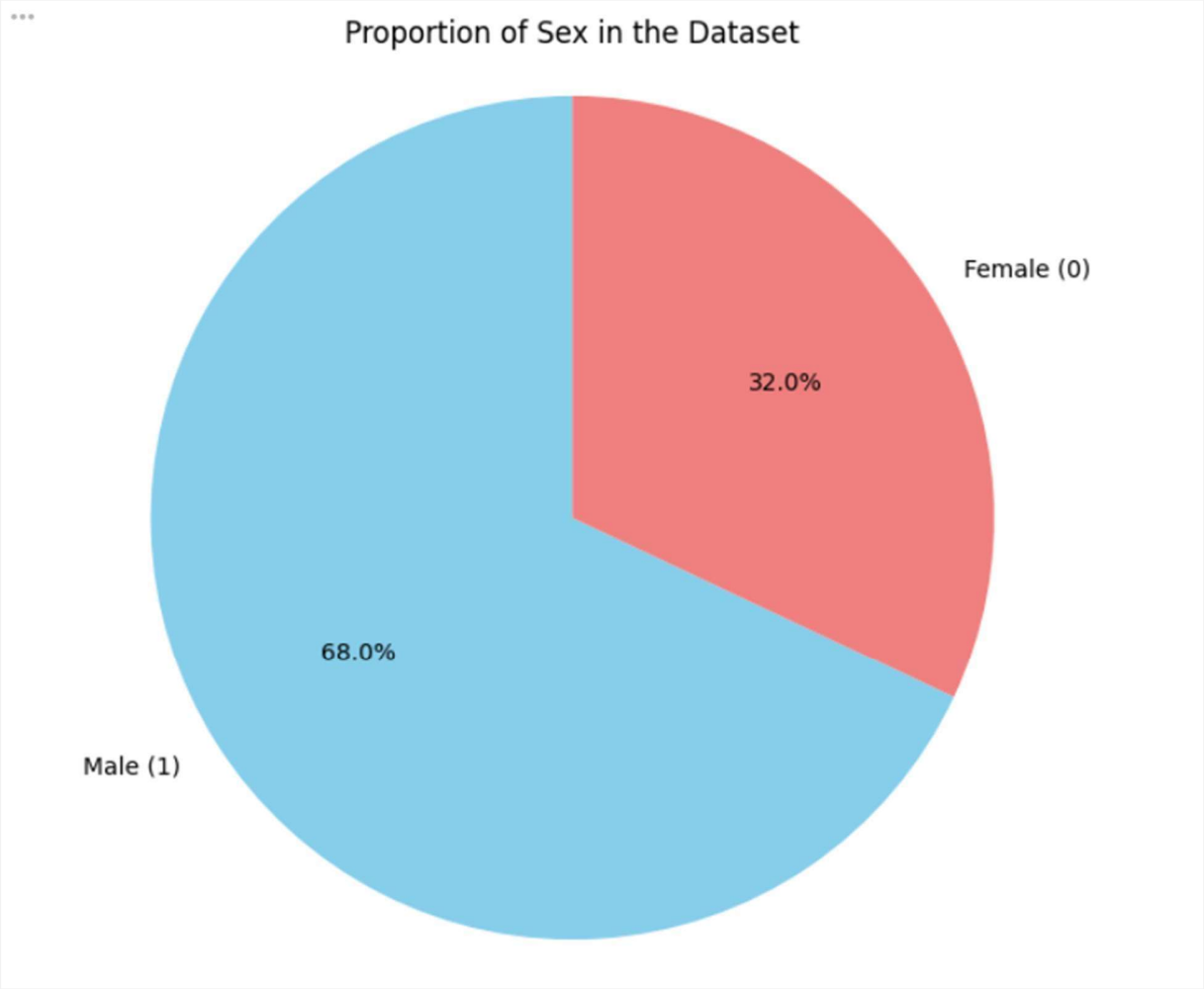
16 Proportion of Heart Disease Presence Results

The distribution shows that the dataset is approximately balanced, with a slight majority of patients having no heart disease (class 0) compared to those with heart disease (class 1). This near-equal distribution is beneficial for model training it means the classifier will not be inherently biased towards always predicting the majority class. Severe class imbalance would require techniques like oversampling (SMOTE) or class weighting to correct, which is not needed here.

3.3.2 Categorical Features

Bar charts and pie charts were generated for all eight categorical features on the original dataset (df) to show the proportion of patients in each category. The key observations are:

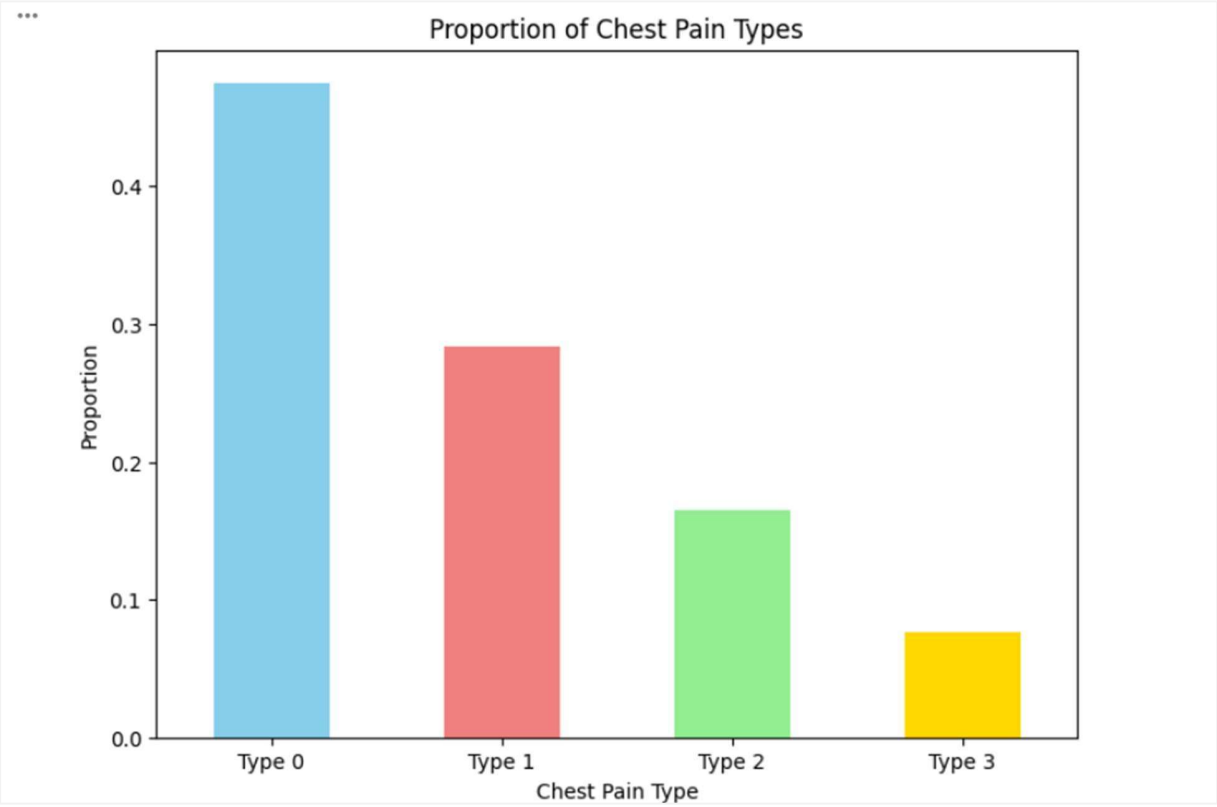
 Sex



17 Proportion of Sex in the Dataset Results

The sex distribution was visualised as a pie chart. The dataset is heavily skewed towards male patients, with males (sex = 1) making up the clear majority (approximately 68%) and females (sex = 0) making up the remaining 32%. This reflects the demographics of the original Cleveland study population and is an important context for interpreting model predictions the model's learned patterns may not generalise equally well across both sexes.

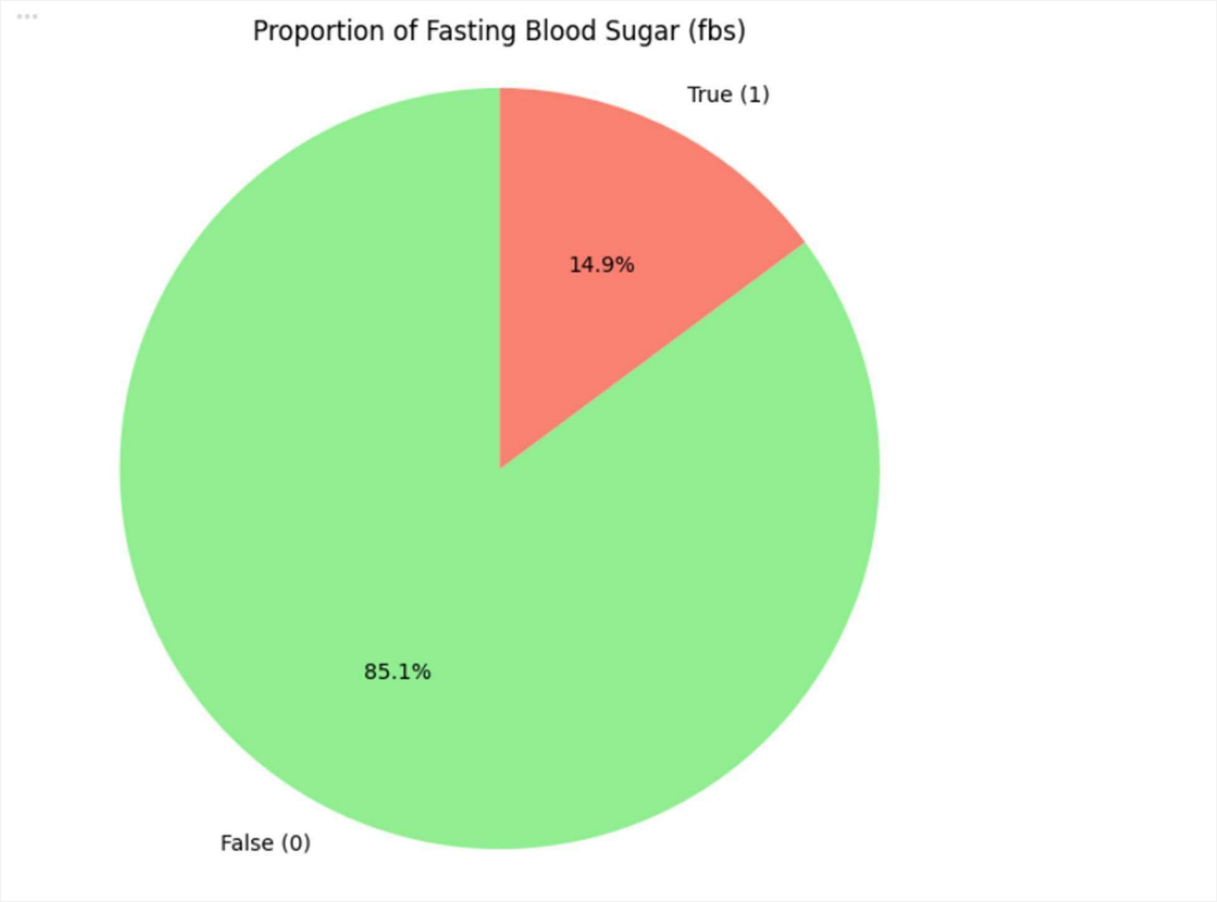
Chest Pain Type (cp)



18 Proportion of Chest Pain Types Results

Chest pain type is one of the most clinically meaningful features in the dataset. Four types are represented: Type 0 (Typical Angina, often associated with heart disease), Type 1 (Atypical Angina), Type 2 (Non-Anginal Pain), and Type 3 (Asymptomatic no visible pain but disease may still be present). The bar chart shows that Asymptomatic (Type 0) is the most prevalent type, which is notable because it is counterintuitively associated with higher heart disease risk. Type 2 (NonAnginal Pain) is the second most common.

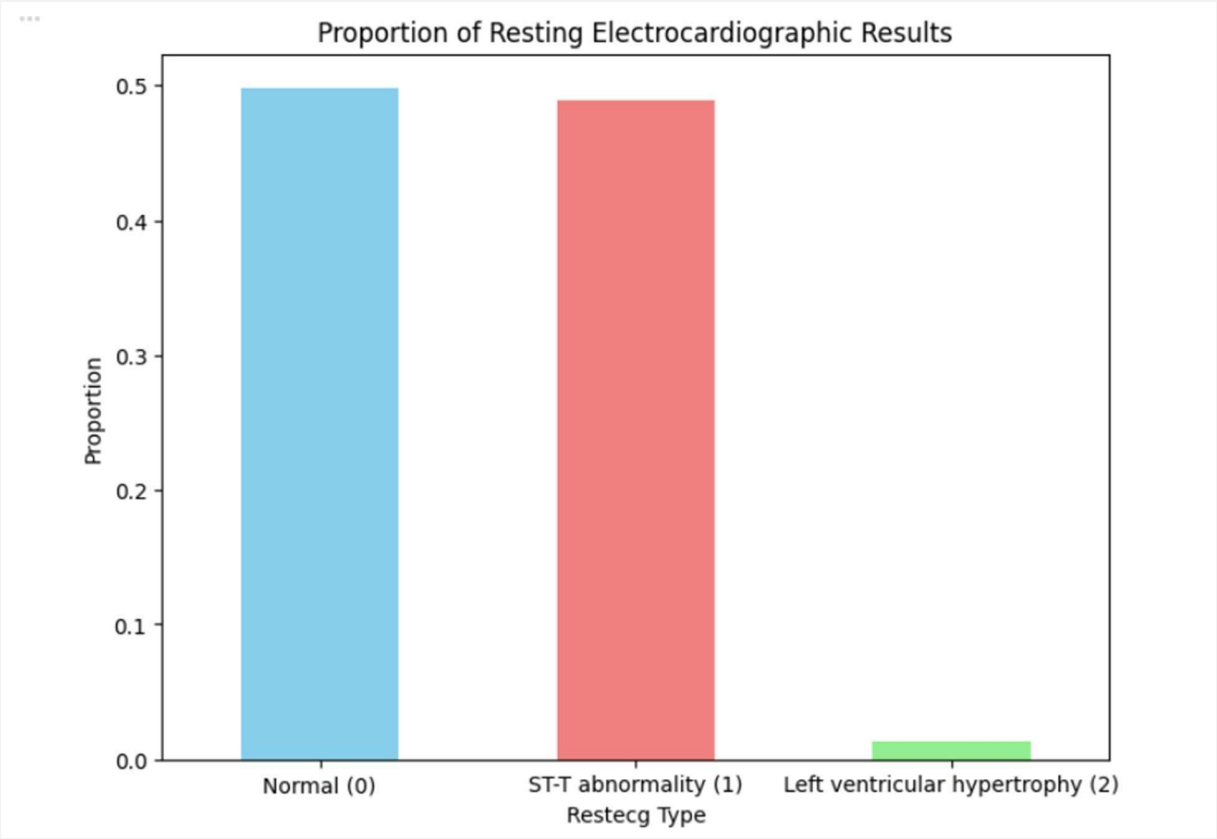
Fasting Blood Sugar (fbs)



19 Proportion of Fasting Blood Sugar Results

Fasting blood sugar was visualised as a pie chart. The vast majority of patients (approximately 85%) have normal fasting blood sugar levels (fbs = 0, meaning blood sugar ≤ 120 mg/dl). Only a small fraction (approximately 15%) have elevated blood sugar (fbs = 1). This high imbalance within the feature itself means fbs alone may not be a strong discriminator between disease classes.

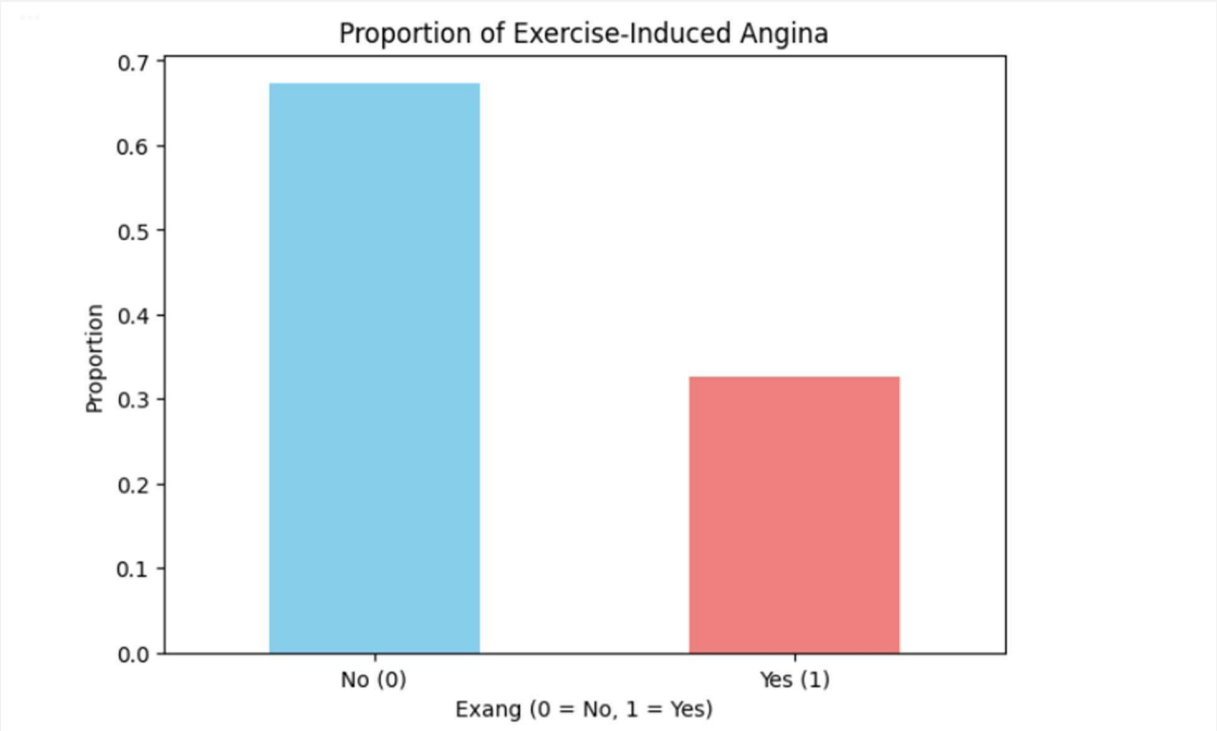
Resting ECG (restecg)



20 Proportion of Resting Electrocardiographic Results

The resting ECG results show that the majority of patients recorded either a normal ECG (restecg = 0) or an ST-T wave abnormality (restecg = 1), which can indicate underlying cardiac issues. A smaller proportion exhibited left ventricular hypertrophy (restecg = 2), which is associated with more serious heart conditions. The bar chart illustrates this clearly, with Type 0 being most prevalent, followed by Type 1.

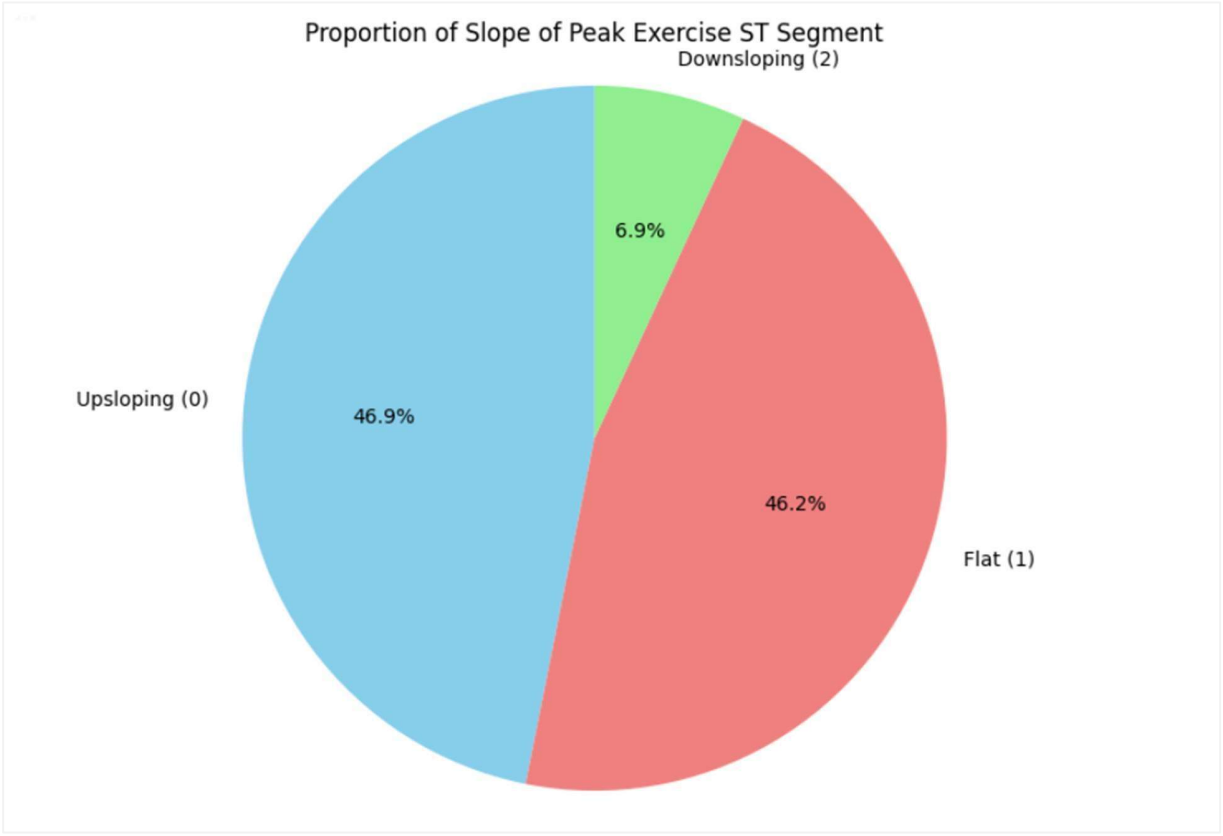
Exercise-Induced Angina (exang)



21 Proportion of Exercise-Induced Angina Results

Exercise-induced angina is a binary feature indicating whether a patient experienced chest pain during exercise. The majority of patients did not experience angina during exercise (exang = 0). However, those who did (exang = 1) represent a clinically important subgroup, as exercise-induced angina is a well-established risk indicator for coronary artery disease (Detrano, 1989)

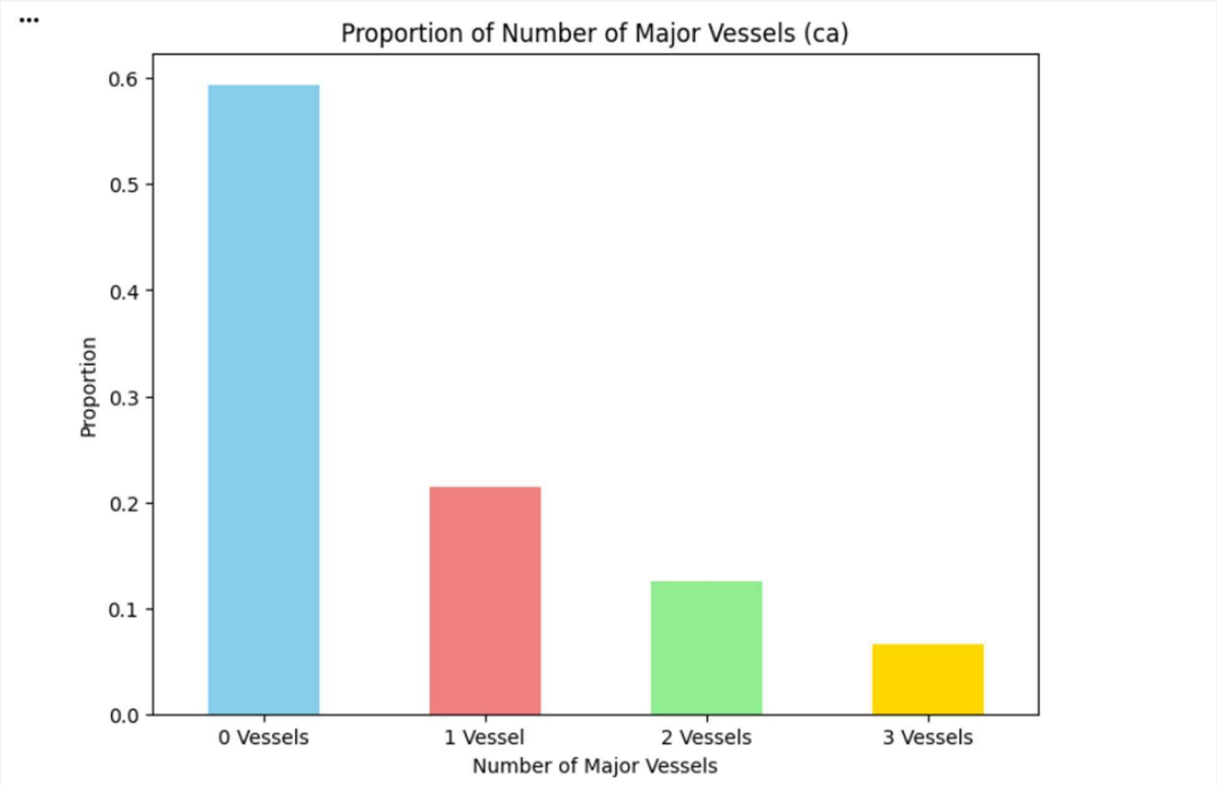
Slope of the ST Segment (slope)



22 Proportion of slope of peak exercise

The slope feature describes the shape of the ST segment during peak exercise. It is visualised as a pie chart. Upsloping (slope = 0) is generally considered normal, flat (slope = 1) may indicate moderate risk, and downsloping (slope = 2) is strongly associated with heart disease. The chart shows that upsloping is the most common pattern in this dataset, with a meaningful proportion also showing a flat slope.

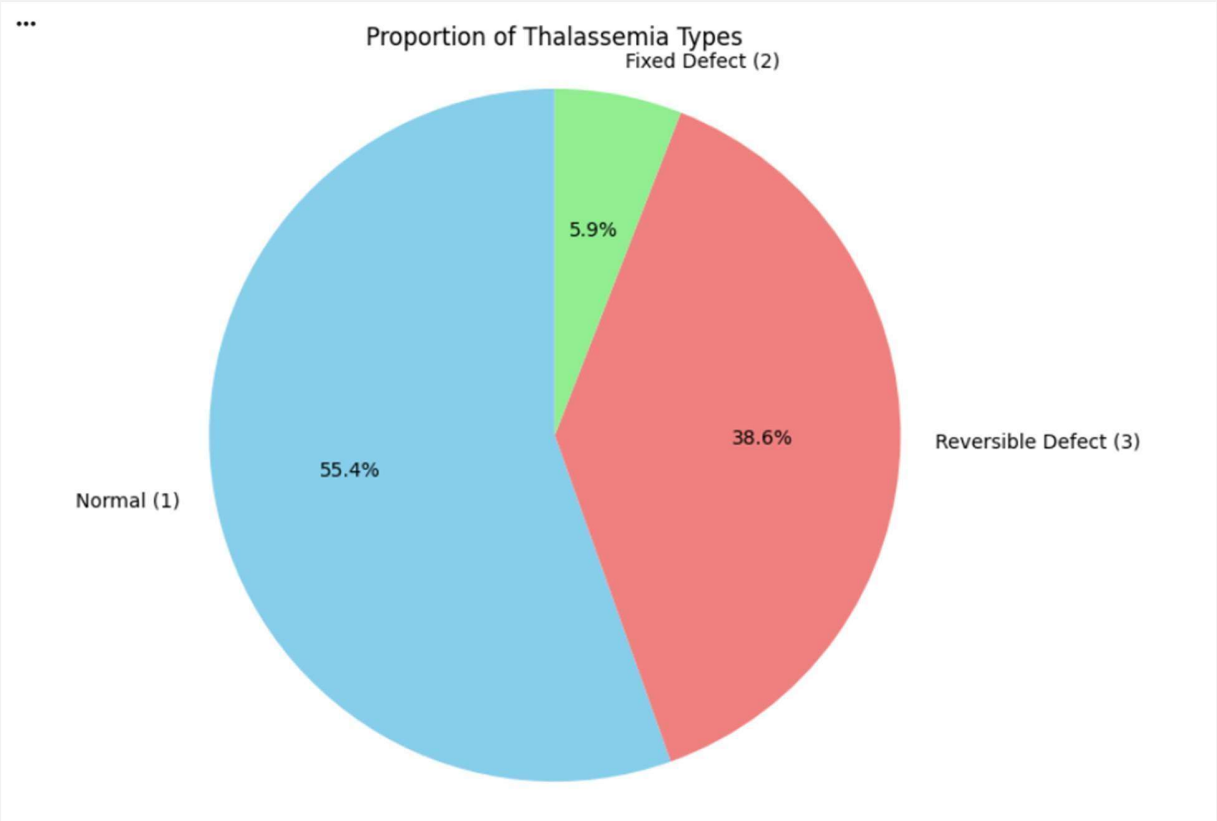
Number of Major Vessels (ca)



23 Proportion of Number of major vessels

The ca feature counts the number of major blood vessels coloured during fluoroscopy (a type of X-ray imaging). A higher number of coloured vessels generally indicates more severe disease. The bar chart shows that the majority of patients have zero coloured vessels (ca = 0), meaning their vessels are relatively clear. The count decreases as the number of vessels increases, reflecting the clinical expectation that multi-vessel disease is less common.

Thalassemia (thal)



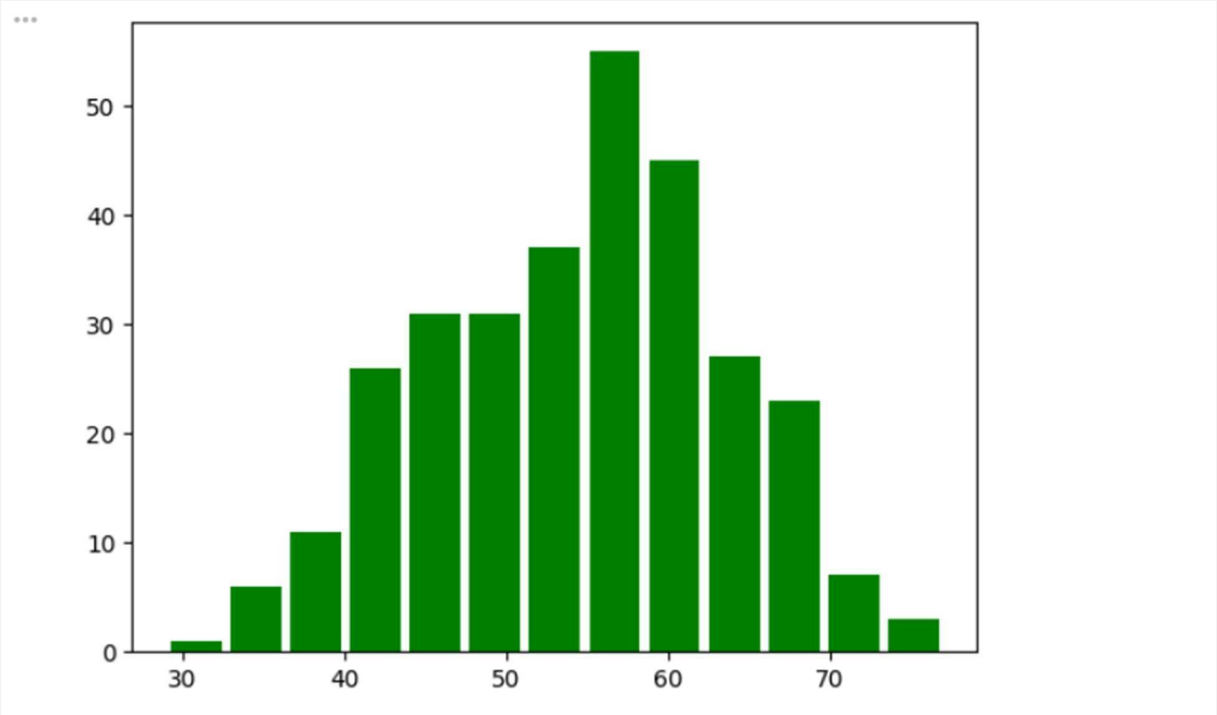
24 Proportion of Thalassemia Types

Thalassemia type is shown as a pie chart. The three categories present are Normal (thal = 1), Fixed Defect (thal = 2, meaning permanent blood flow abnormality), and Reversible Defect (thal = 3, meaning the abnormality appears under stress but not at rest). Reversible defects are clinically significant as they are often associated with ischaemia (restricted blood flow). The pie chart shows that Normal and Reversible Defect are the dominant categories.

3.3.3 Numerical Features

Descriptive statistics (using describe()) and distribution plots (histograms and box plots) were generated for the five continuous numerical features. These were examined on the original dataset (df) and the cleaned dataset (df_cleaned) to show the data's natural distribution before standardisation.

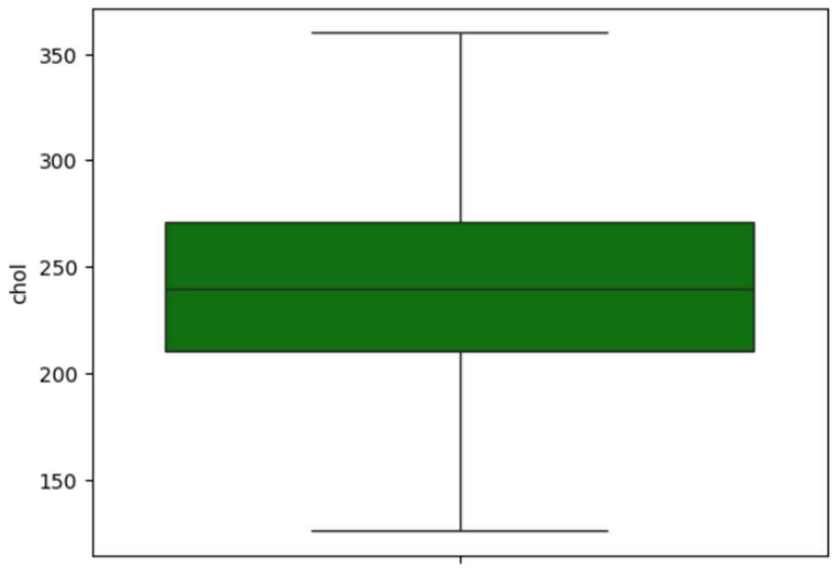
- Age



25 Distribution of age

The age histogram shows a roughly bell-shaped (approximately normal) distribution centred around the mid-50s. This reflects the clinical context heart disease is more prevalent in middleaged and older adults. The majority of patients in the dataset are between 40 and 65 years old, with the distribution tapering at both the younger and older extremes. The describe() output confirms: minimum age = 29, maximum age = 77, mean ≈ 54.

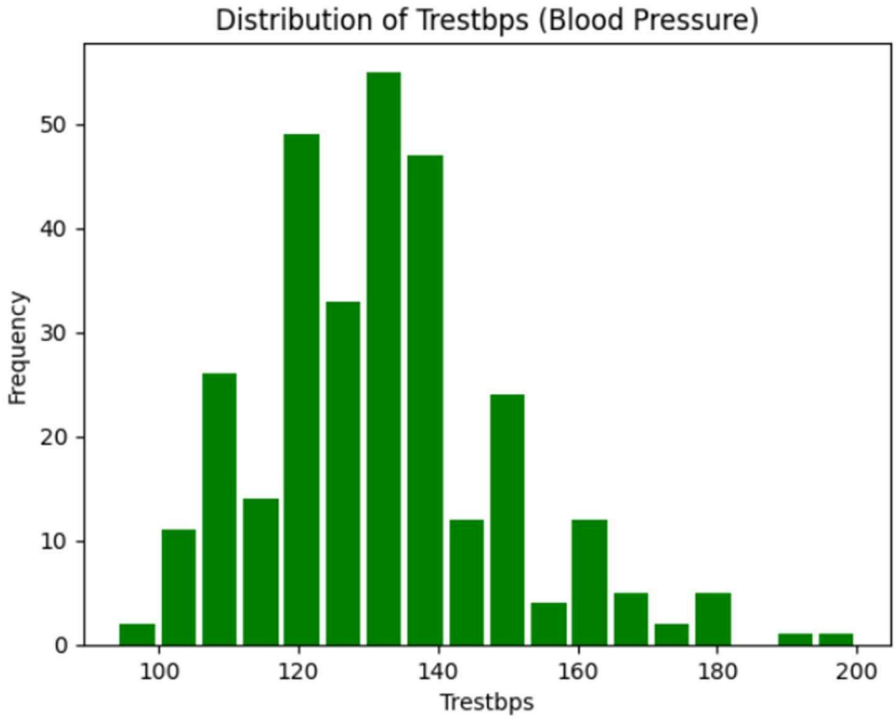
 Serum Cholesterol (chol)



26 Boxplot of cholesterol levels (chol)

The cholesterol box plot on df_cleaned shows a relatively compact distribution after outlier removal, with most values falling within a moderate clinical range. The describe() output on the original df shows a mean around 246 mg/dl. Cholesterol distributions in medical datasets are typically right-skewed most patients cluster around moderate levels, but a small number have very high values. The removal of outliers (5 rows) helped tighten this distribution.

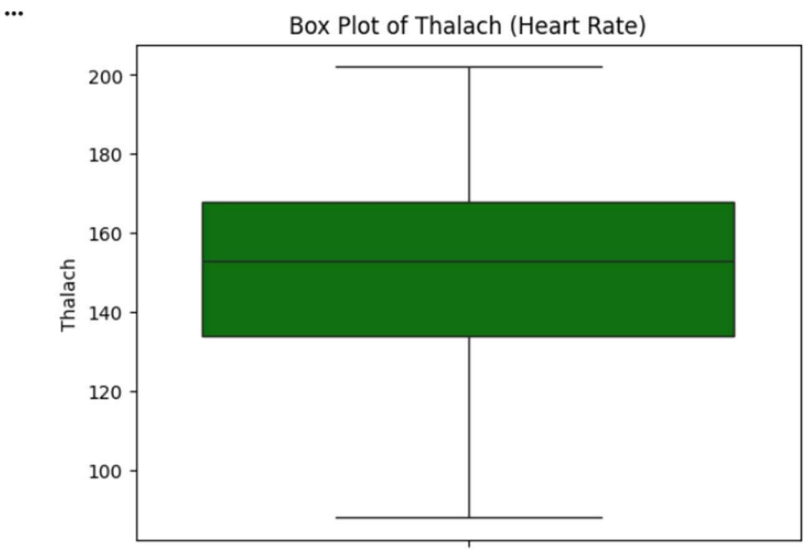
 Resting Blood Pressure (trestbps)



27 Distribution of Blood Pressure

The histogram of trestbps shows a right-skewed distribution, with the bulk of patients recording resting blood pressures between 110 and 160 mm Hg within or slightly above the normal clinical range. The 9 outliers removed during preprocessing were predominantly high-value readings, which, if retained, could have inflated the feature's apparent range and skewed model learning.

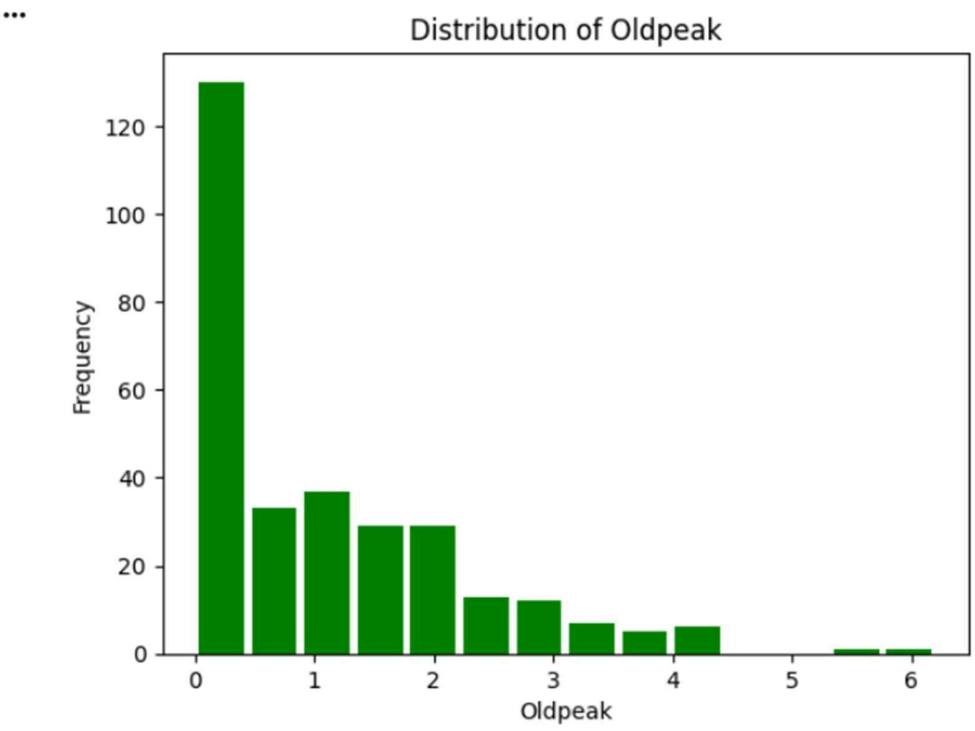
 Maximum Heart Rate (thalach)



28 Boxplot of Thalach (Heart Rate)

The box plot of thalach (on df_cleaned) shows a distribution that is roughly symmetric, centred around 150 beats per minute. Maximum heart rate tends to decrease with age, making it an important predictor of cardiovascular fitness. Only 1 outlier was removed from this feature, indicating the data was relatively clean. The cleaned distribution shows a tighter, more reliable range for model training.

 ST Depression (oldpeak)



29 Distribution of Oldpeak

The oldpeak histogram shows a strong right skew the majority of patients have low ST depression values (close to 0), while a smaller number have substantially elevated values. This skew is clinically expected, as significant ST depression occurs in fewer patients but is strongly associated with heart disease. Five outliers were removed from this feature during preprocessing.

3.4 Bivariate Analysis

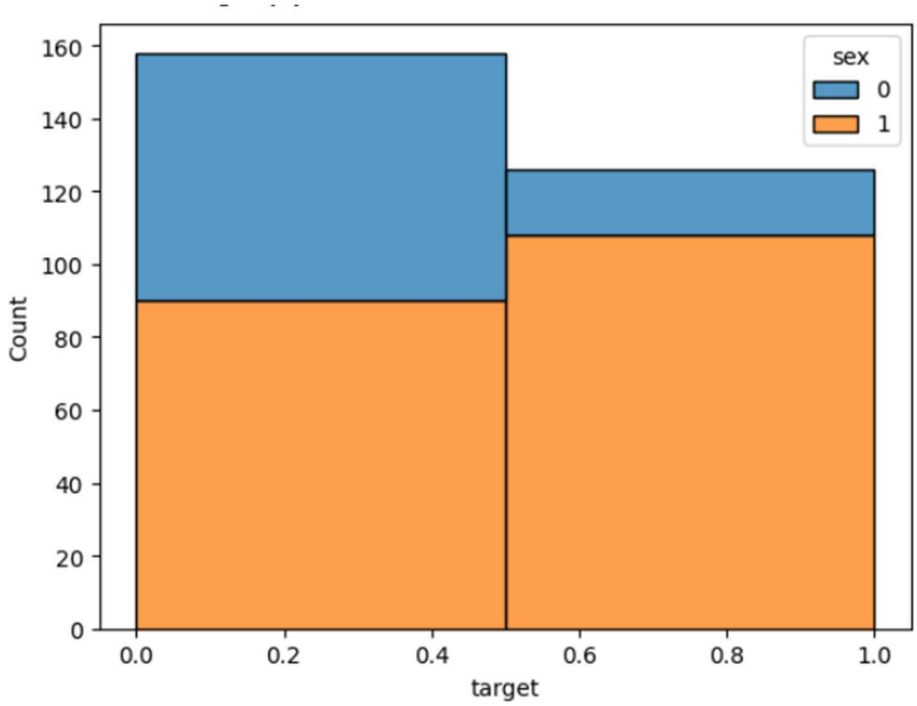
Bivariate analysis examines the relationship between two variables at a time. The primary focus here is on understanding how each feature relates to the target variable (heart disease presence), which directly informs which features are likely to be useful for prediction.

3.4.1 Categorical Features vs Target

Stacked histograms and grouped count plots were created using sns.histplot() and sns.countplot() to compare the distribution of each categorical feature across the two target classes (0 = No Disease, 1 = Disease). Cross-tabulations were also computed using pd.crosstab(), normalised by the total number of rows, to show proportions:

```
round(pd.crosstab(df_cleaned['sex'], df_cleaned['target']) / len(df_cleaned), 2)
```

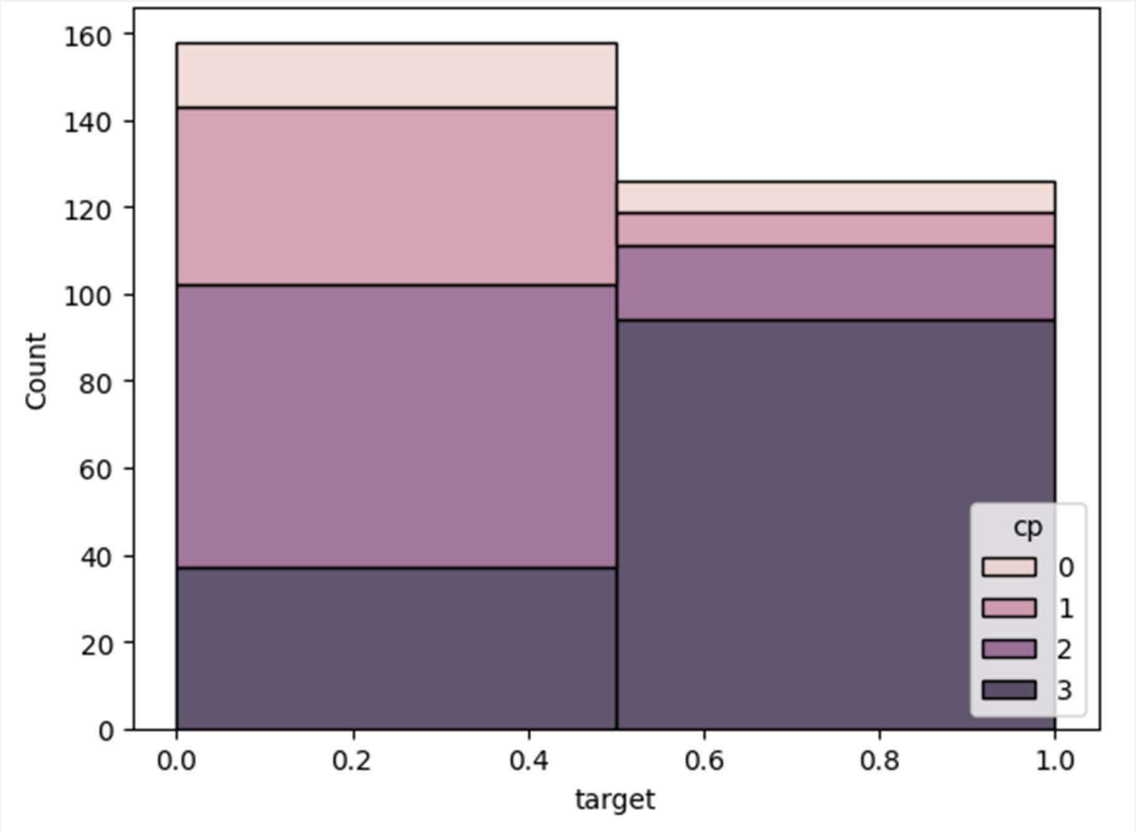
- Sex vs. Target:



30 Distribution of heart disease by gender

The stacked histogram reveals that males (sex = 1) make up the majority of both disease and nodisease groups, simply because they are more represented in the dataset overall. However, looking at proportions within each sex group, a greater proportion of female patients are classified as having heart disease compared to male patients, suggesting that sex interacts with disease presence in a meaningful way. This makes sex a relevant feature for the model.

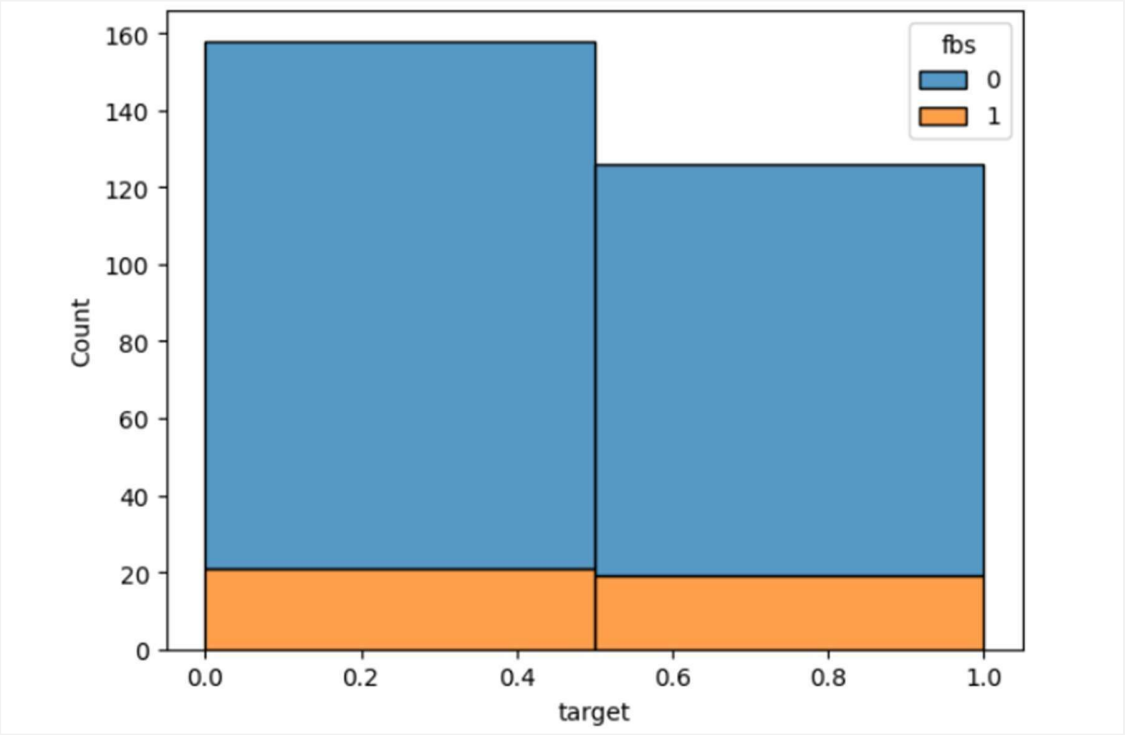
- Chest Pain vs. Target:



31 Distribution of heart disease by chest pain type

The chest pain type is one of the strongest visual separators between classes. Patients with Type 0 (Typical Angina) and Type 3 (Asymptomatic) show a noticeably higher proportion of heart disease cases compared to Types 1 and 2. This aligns with clinical understanding typical angina and asymptomatic patients with underlying disease are key presentations of coronary artery disease.

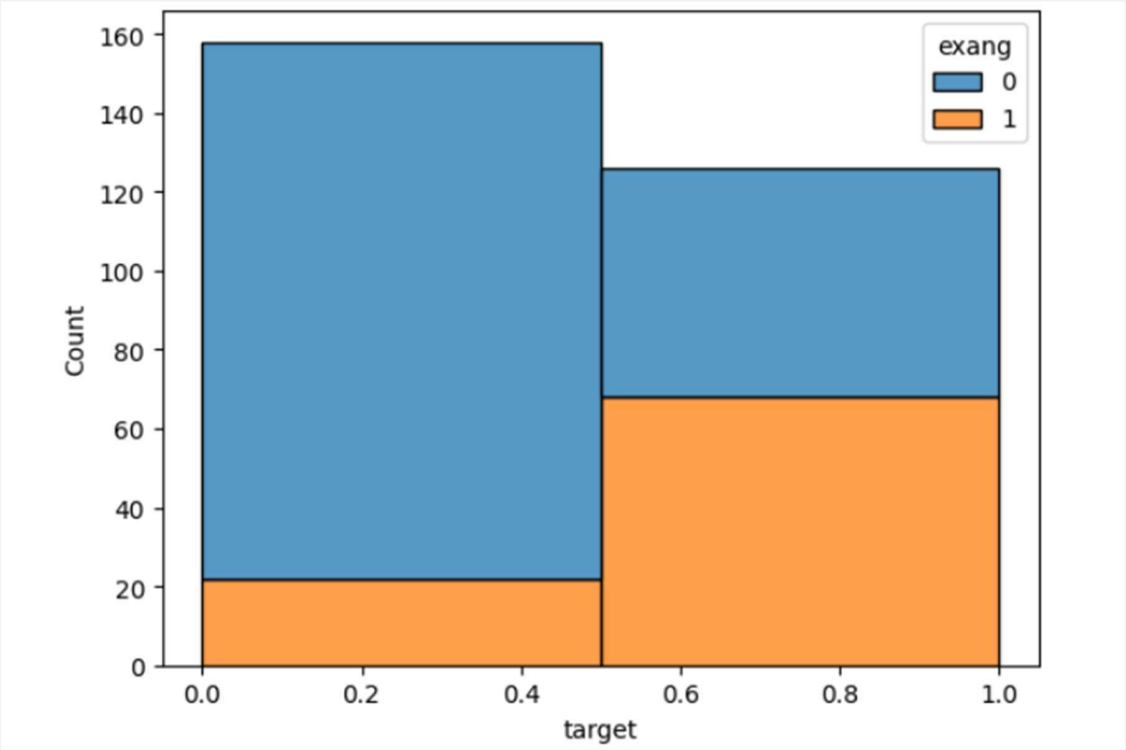
- Fasting Blood Sugar vs. Target:



32 Distribution of heart disease by fasting blood sugar (fbs)

The cross-tabulation and stacked histogram show that fasting blood sugar has limited discriminating power between the two classes. Both high and normal fbs groups show similar proportions of heart disease, suggesting this feature alone may not be a strong predictor. However, it may still contribute in combination with other features.

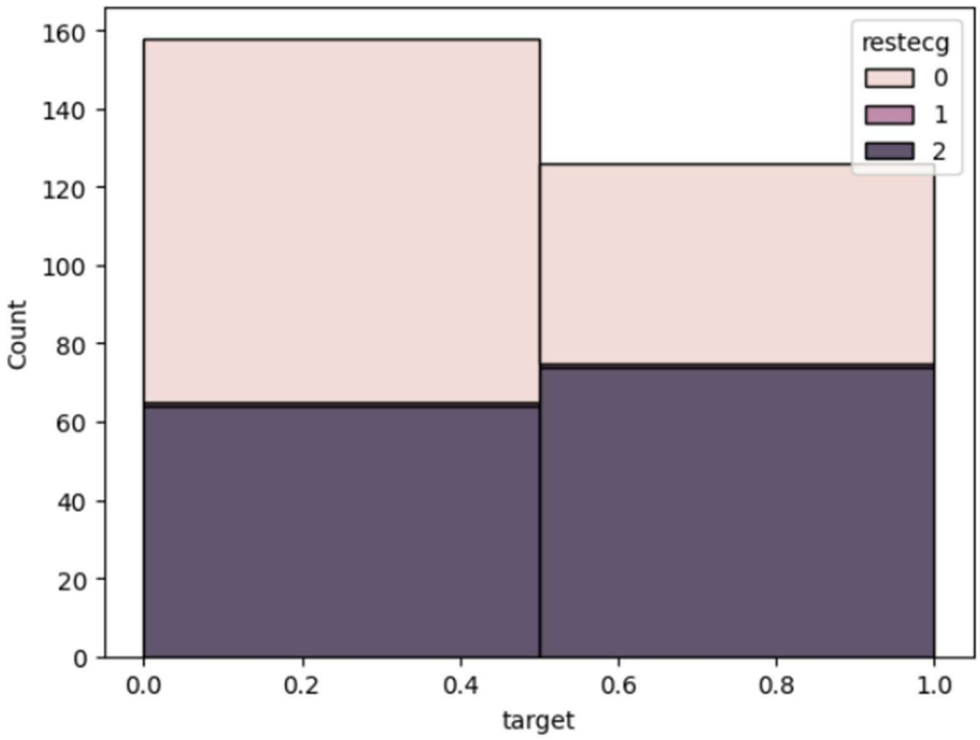
- Exercise-Induced Angina vs. Target:



33 Distribution of heart disease by exercise-induced angina (exang)

This is one of the clearest bivariate relationships in the dataset. Patients who experienced exerciseinduced angina (exang = 1) have a substantially higher proportion of heart disease cases compared to those who did not (exang = 0). This makes exang one of the more diagnostically significant features in the dataset, consistent with its clinical relevance as a screening criterion.

 Resting ECG vs. Target:



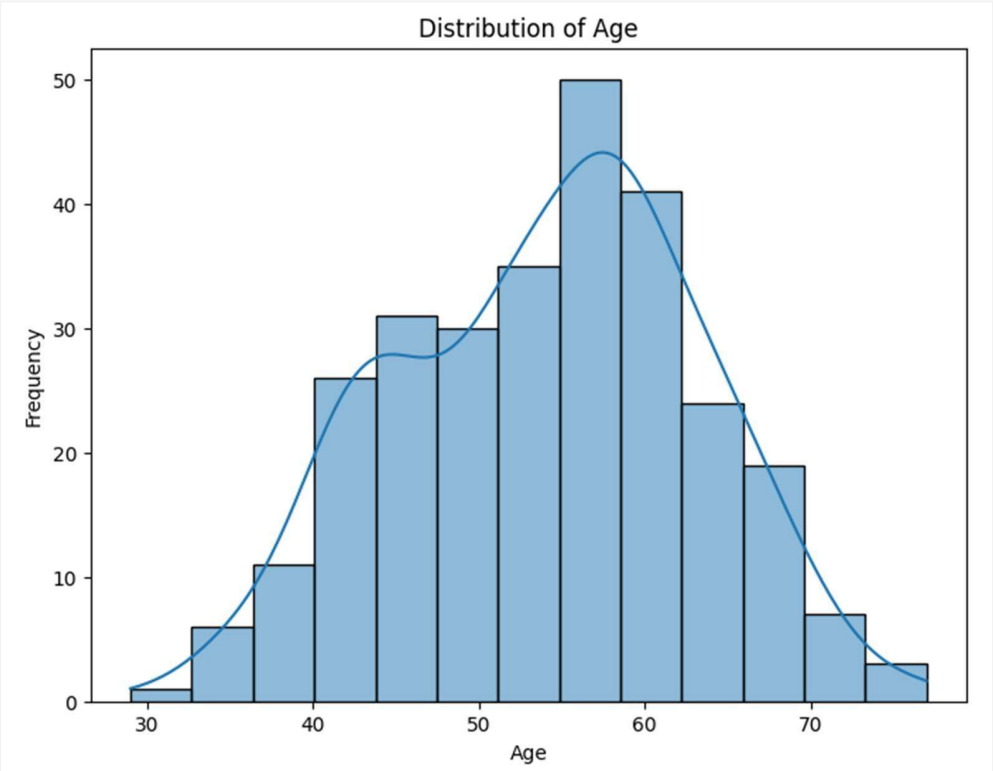
34 Distribution of heart disease by resting ECG (restecg)

The grouped distribution shows that patients with ST-T wave abnormality (restecg = 1) and left ventricular hypertrophy (restecg = 2) tend to have a higher proportion of heart disease compared to those with a normal ECG (restecg = 0). However, the difference is not as dramatic as with chest pain type or exercise-induced angina.

3.4.2 Histograms for Numerical Features by Target

Histograms with kernel density estimation (KDE) were plotted for each of the five numerical features on df_cleaned to show how the distribution differs between patients with and without heart disease:

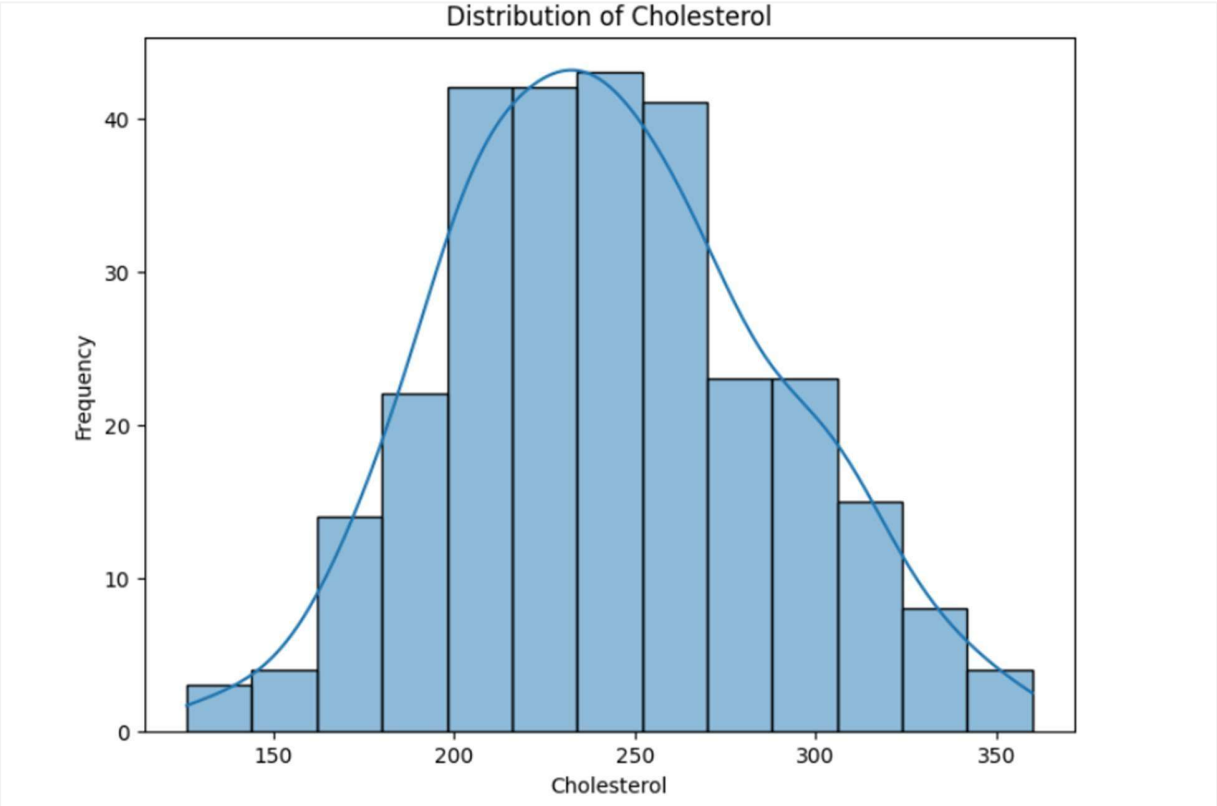
- Age:



35 Distribution of age (numerical feature)

The age distributions for both classes overlap considerably, though older patients tend to be slightly more represented in the heart disease group. Age alone is not a strong discriminator, but it becomes more powerful when combined with other clinical features.

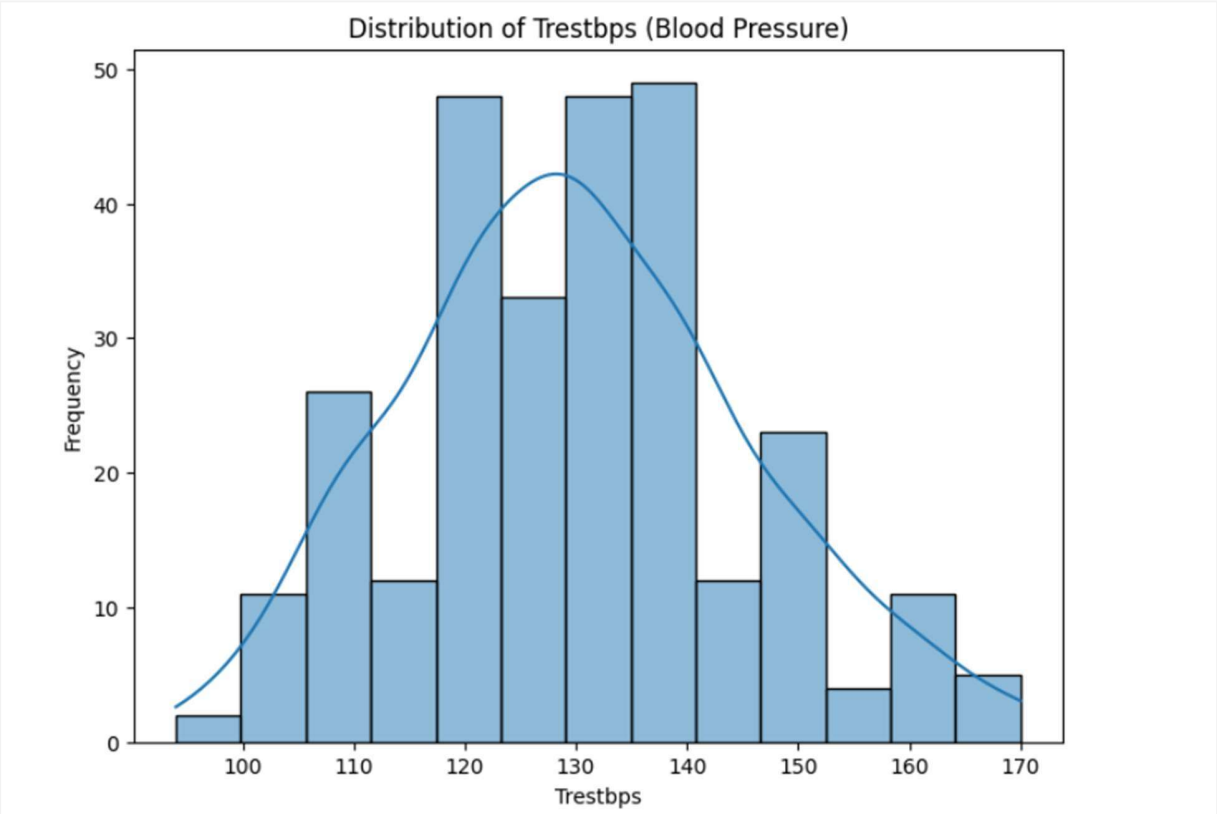
- Cholesterol:



36 Distribution of Cholesterol(numerical feature)

Interestingly, cholesterol does not show a strong separation between the two classes after outlier removal. Patients in both groups have broadly similar cholesterol distributions, suggesting that cholesterol alone may have limited predictive value in this dataset a finding that has been noted in other studies using the Cleveland dataset (Detrano et al., 1989).

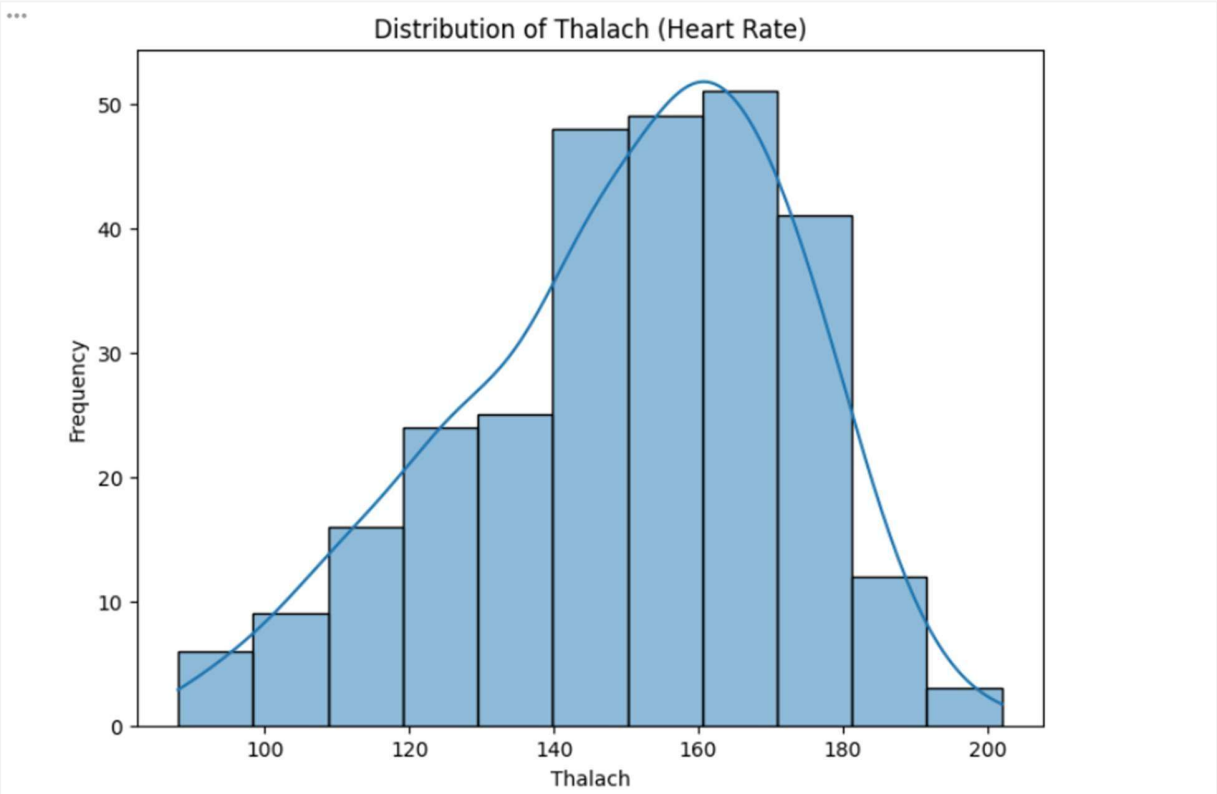
- Blood Pressure (trestbps):



37 Distribution of Trestbps (numerical feature)

The blood pressure distributions for both classes are similar, with a slight tendency for higher values among disease-positive patients. The distributions are not clearly separable, suggesting trestbps is a weak individual predictor but may contribute as part of a feature combination.

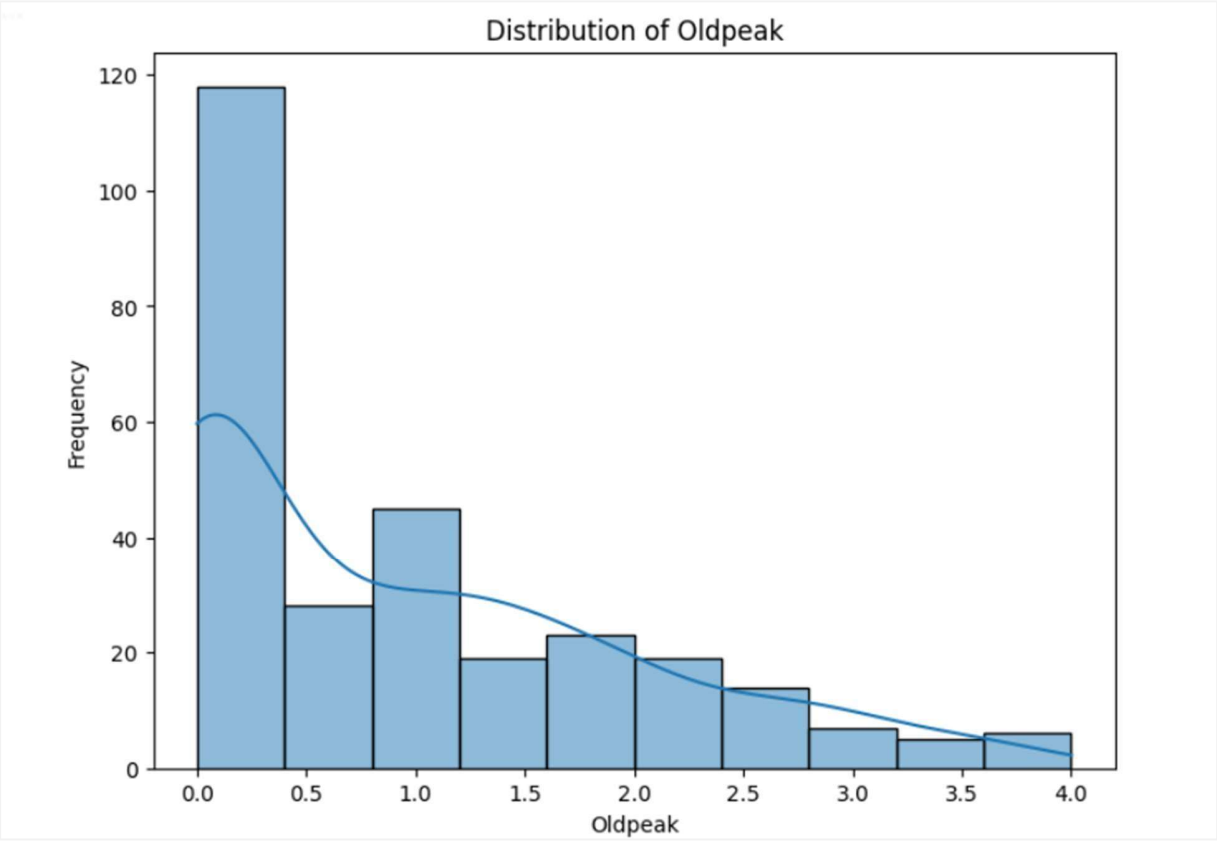
- Maximum Heart Rate (thalach):



38 Distribution of Thalach (numerical feature)

This is one of the most visually informative distributions. Patients without heart disease tend to achieve higher maximum heart rates, while patients with heart disease cluster at lower values. This inverse relationship between maximum heart rate and disease presence is clinically wellestablished and makes thalach one of the stronger predictors in the dataset.

- ST Depression (oldpeak):



39 Distribution of Oldpeak (numerical feature)

The oldpeak distributions show clear separation between the two classes. Patients without heart disease are heavily concentrated near 0, while patients with heart disease are more spread across higher values. This makes oldpeak one of the most discriminative numerical features for heart disease classification.

3.4.3 Pair Plot

A pair plot was generated for the five numerical features (age, trestbps, chol, thalach, oldpeak) from df_cleaned, with points coloured by the target class using the Viridis palette. The diagonal of the pair plot shows KDE distributions for each individual feature, while the off-diagonal panels show scatter plots between all pairs of features

```
# We'll include the 'target' variable to see how it relates to these numerical features
sns.pairplot(df_cleaned, vars=selected_numerical_cols, hue='target', palette='viridis', diag_kind='kde')
```



40 Histograms and Pair Plot

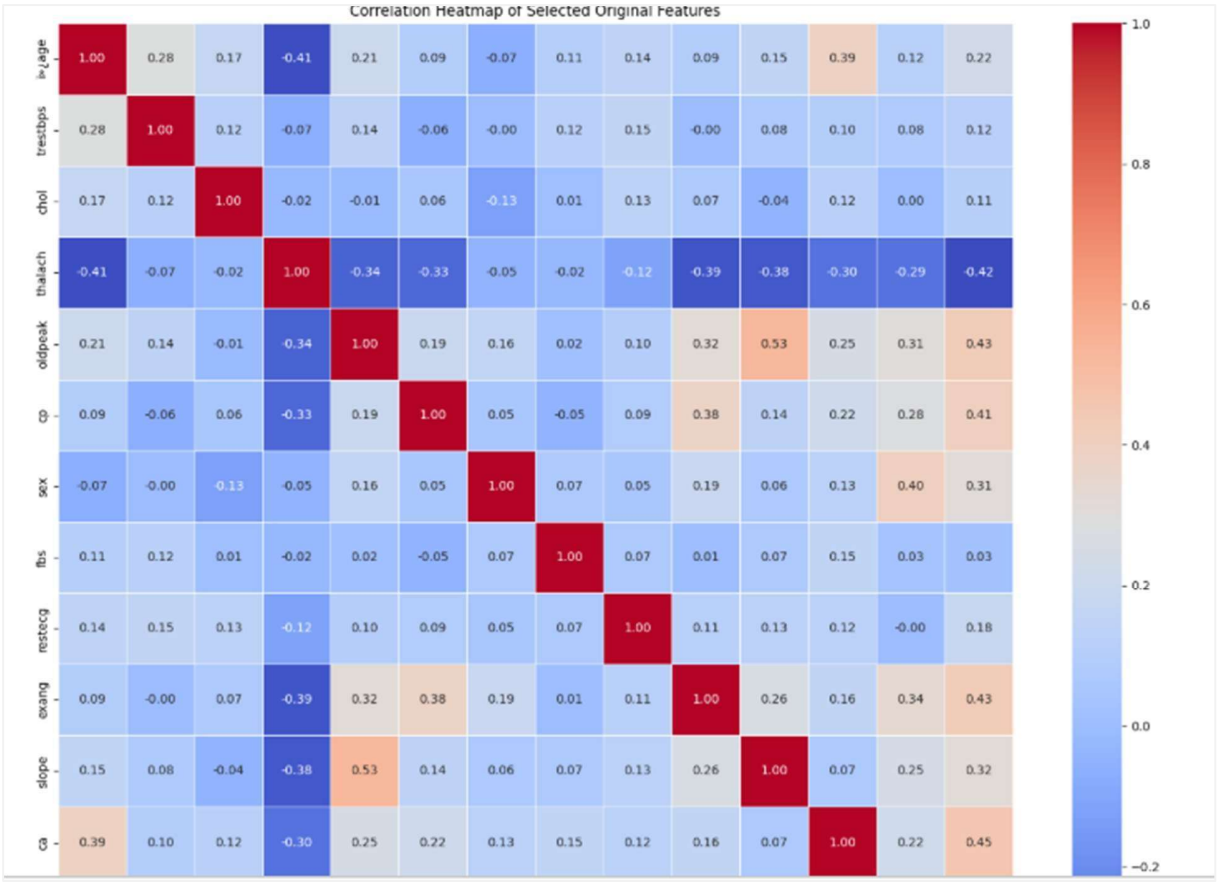
The pair plot reinforces the earlier observations: thalach and oldpeak show the clearest separation between the two classes across both their individual distributions and their scatter plots with other features. The age-thalach scatter plot, for instance, shows a visible downward trend older patients with lower maximum heart rates are more often classified as disease-positive. Features like chol and trestbps show more overlap between classes, confirming their weaker individual discriminating power.

3.5 Correlation Heatmap

The final step in feature exploration was computing and visualising a correlation matrix for the selected original features (before one-hot encoding) from df_cleaned. Pearson correlation

coefficients measure the linear relationship between each pair of features, ranging from -1 (perfect negative correlation) to +1 (perfect positive correlation). Values near 0 indicate no linear relationship.

```
selected_cols_for_corr = [
    'i»¿age', 'trestbps', 'chol', 'thalach', 'oldpeak', # Original numerical features
    'cp', 'sex', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal', # Original integer-encoded categorical features
    'target' # Target variable
]
```



41 Correlation Heatmap

The heatmap uses a coolwarm colour scheme deep red represents strong positive correlations, deep blue represents strong negative correlations, and white/light colours represent weak or no correlation. Each cell in the heatmap shows the Pearson coefficient rounded to two decimal places. The most important correlations with the target variable are:

- thalach (Maximum Heart Rate): Shows a negative correlation with target (approximatel- 0.42), meaning patients with higher maximum heart rates tend not to have heart disease. This

confirms the bivariate histogram finding and marks thalach as the strongest negatively correlated numerical predictor.

- oldpeak (ST Depression): Shows a positive correlation with target (approximately +0.43), meaning higher ST depression values are associated with greater heart disease presence. This is the strongest positively correlated numerical feature.
- cp (Chest Pain Type): Shows a negative correlation with target (approximately -0.43), indicating that higher chest pain type codes (as coded in this dataset) are associated with less disease. This reflects the dataset's coding scheme where Type 0 (Typical Angina) is associated with disease but coded as the lowest value.
- ca (Number of Vessels): Shows a positive correlation with target (approximately +0.46), suggesting that a higher number of fluoroscopy-coloured vessels is associated with greater disease severity.
- exang (Exercise-Induced Angina): Shows a positive correlation with target (approximately +0.43), consistent with the bivariate analysis finding.

Among the features themselves (inter-feature correlations):

- age and thalach show a moderate negative correlation (approximately -0.40), reflecting the well-known clinical fact that maximum heart rate declines with age.
- oldpeak and slope are moderately correlated (approximately +0.58), as ST depression and ST segment slope are both measures of the same underlying physiological signal during exercise stress.
- Most other feature pairs show low correlations (absolute values < 0.30), indicating limited multicollinearity in the dataset. This is a positive finding it means most features are providing relatively independent information to the model, reducing the risk of redundancy.

The heatmap effectively provides a visual summary of which features are most relevant for predicting heart disease and helps identify any redundant features that might be candidates for removal in later stages of model development.

4 Traditional Machine Learning Models

4.1 Introduction

With the data fully preprocessed and explored in Parts 1 and 2, the next step is to train and evaluate traditional machine learning (ML) models to classify patients as having heart disease (1) or not (0). This section covers six models: Logistic Regression, Random Forest, Support Vector Machine (SVM), Decision Tree, AdaBoost, and XGBoost. Each model is first trained with default settings (baseline), then optimised through hyperparameter tuning. All models are evaluated using five metrics Accuracy, Precision, Recall, F1-Score, and ROC-AUC and their confusion matrices, overfitting behaviour, and feature importances are discussed.

4.2 Data Splitting

Before any model is trained, the preprocessed dataset (df_encoded) was divided into features (X) and the target label (y). The data was then split into a training set and a test set using scikit-learn's train_test_split() function (Pedregosa, et al., 2011):

```
# Define features (X) and target (y)
X = df_encoded.drop('target', axis=1)
y = df_encoded['target']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Figure 42 Column selection list for pair plot analysis

The stratify=y argument ensures that the proportion of disease and no-disease patients is preserved in both the training and test sets, which is particularly important in medical datasets to avoid a biased evaluation (Géron, 2022).The resulting split was:

- X_train: 228 samples × 30 features (80% of the data used for training)
- X_test: 57 samples × 30 features (20% of the data used for evaluation)

The choice of an 80/20 split is a widely used standard that provides enough training data for the model to learn patterns while holding back a sufficient unseen test set for reliable performance evaluation.

4.3 Evaluation Metrics

Five metrics were used to evaluate every model. Understanding why each metric matters especially in a medical context is essential:

- Accuracy measures the overall proportion of correct predictions. It is a useful general indicator but can be misleading if classes are imbalanced.
- Precision measures, of all the patients predicted to have heart disease, how many actually do. High precision means fewer false alarms.
- Recall (Sensitivity) measures, of all patients who actually have heart disease, how many were correctly identified. In clinical screening, this is the most critical metric missing a true disease case (a False Negative) is far more dangerous than a false alarm (Kohavi, 1995).
- F1-Score is the harmonic mean of Precision and Recall. It is the best single metric when both false positives and false negatives matter, as it penalises models that sacrifice one for the other.
- ROC-AUC measures the model's ability to discriminate between the two classes across all possible classification thresholds. A value of 1.0 is perfect; 0.5 is no better than random guessing.

A custom helper function was written to compute and display all five metrics consistently across all models:

```
def evaluate_model(model_name, model, X_test, y_test, storage_dict, prediction_dict):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    metrics = {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred, zero_division=0),
        'Recall': recall_score(y_test, y_pred, zero_division=0),
        'F1-Score': f1_score(y_test, y_pred, zero_division=0),
        'ROC-AUC': roc_auc_score(y_test, y_proba)
    }
```

Figure 43evaluate_model() helper function definition

4.4 Baseline Models

All six models were first trained with their default scikit-learn settings. This establishes a performance baseline before any optimisation is applied, allowing a fair comparison of how much hyperparameter tuning actually helps.

4.4.1 Model 01 - Logistic Regression

Logistic Regression is one of the simplest and most interpretable classification algorithms. Despite its name, it is a classification model, not a regression one. It works by estimating the probability that a data point belongs to a given class using the logistic (sigmoid) function, which maps any real-valued number to a value between 0 and 1 (Hosmer & Lemeshow, 2000). It was trained using the 'liblinear' solver, which is well-suited for small datasets:

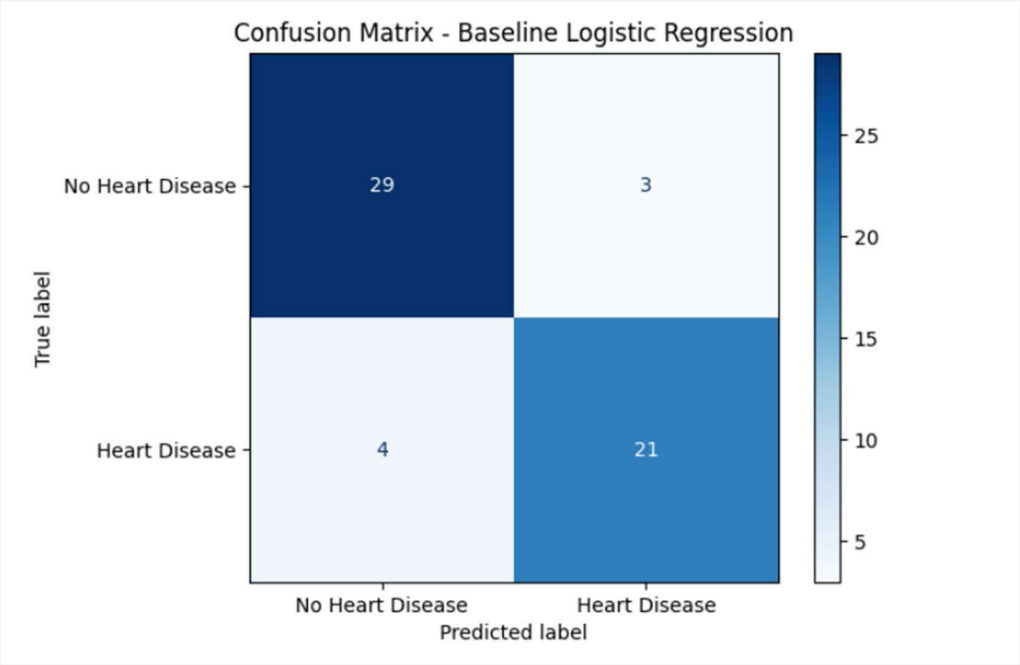
```
log_reg_baseline = LogisticRegression(random_state=42, solver='liblinear')
log_reg_baseline.fit(X_train, y_train)
```

Figure 44 Baseline Logistic Regression training code

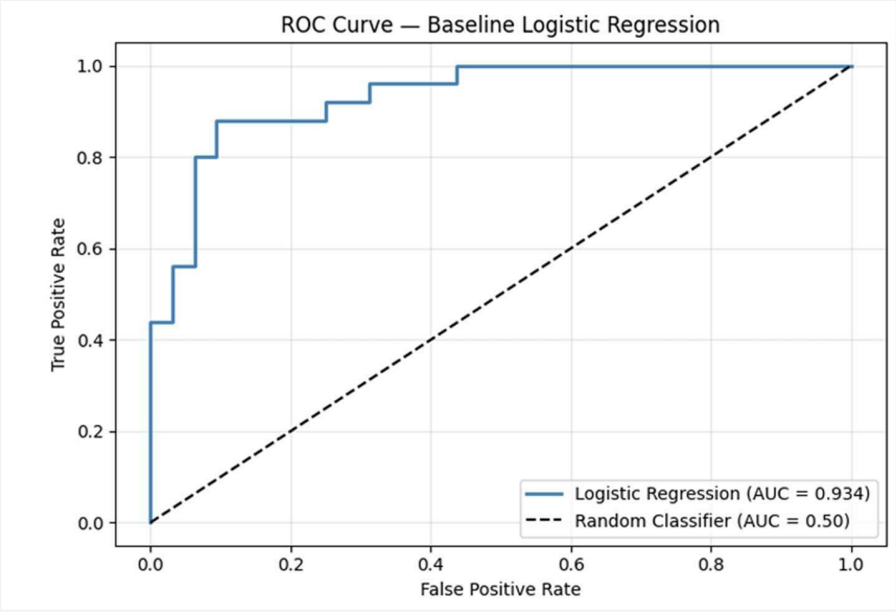
```
Logistic Regression Evaluation Metrics
Accuracy: 0.8772
Precision: 0.8750
Recall: 0.8400
F1-Score: 0.8571
ROC-AUC: 0.9338
```

Figure 45 Logistic Regression baseline evaluation metrics output

Even with default settings, Logistic Regression produced the strongest baseline performance across all six models. Its high ROC-AUC of 0.9338 indicates excellent class separation ability. The Recall of 0.8400 means it correctly identified 84% of actual heart disease patients a clinically meaningful result for a baseline model.



46 Confusion matrix of Logistic Regression model



47 ROC Curve - Logistic Regression

The Logistic Regression model's AUC is 0.9338, which means that it has a 93.4% chance of ranking a disease patient above a healthy patient. It has the highest AUC and is an excellent model for this medical screening task. It performs much better than random guessing (AUC

0.50), and provides the best baseline in this experiment.

The ROC curve for Logistic Regression begins to move upwards (towards the top-left corner) early on in the plot (at low FPR values). This early steep rise indicates that most patients with disease are correctly diagnosed and few false alarms are triggered. It slows down at higher FPR values and remains at high TPR.

4.4.2 Model 02 - Random Forest Classifier

Random Forest is an ensemble method that builds many individual decision trees during training and combines their predictions through majority voting (Breiman, 2001). By averaging across many trees, it reduces the variance problem of a single decision tree, making it far more stable and generalising better to unseen data. It was trained with default settings (100 trees):

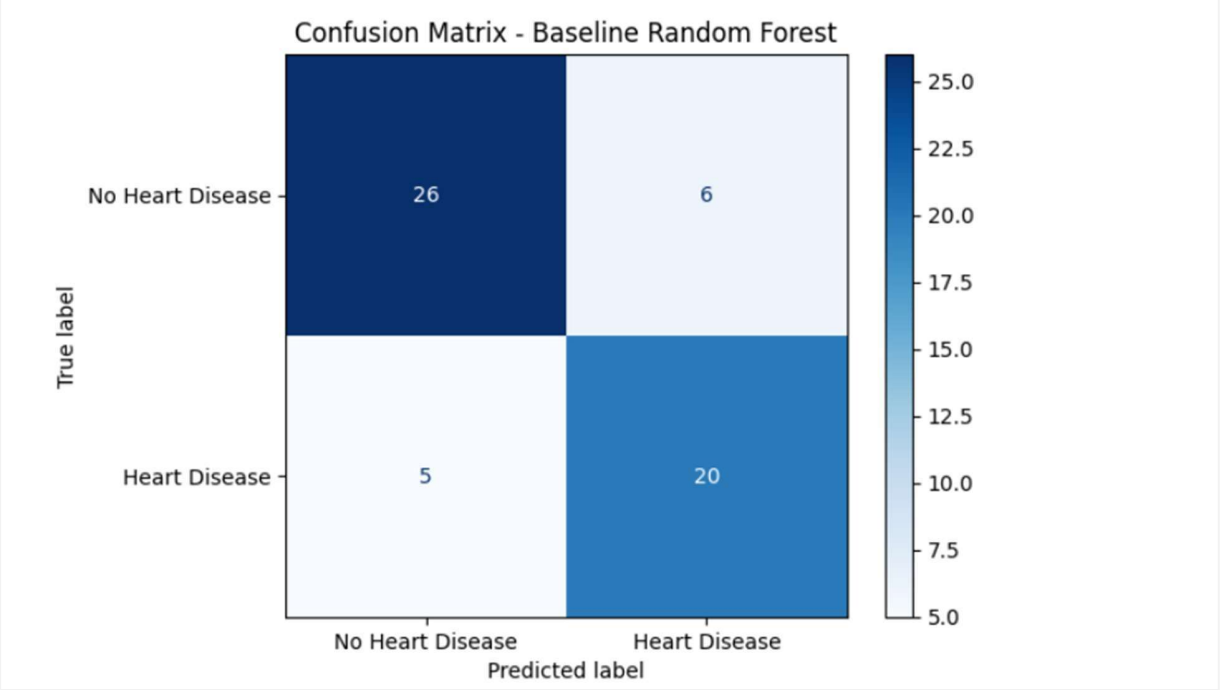
```
rf_baseline = RandomForestClassifier(random_state=42)
rf_baseline.fit(X_train, y_train)
```

Figure 48 Baseline Random Forest training code

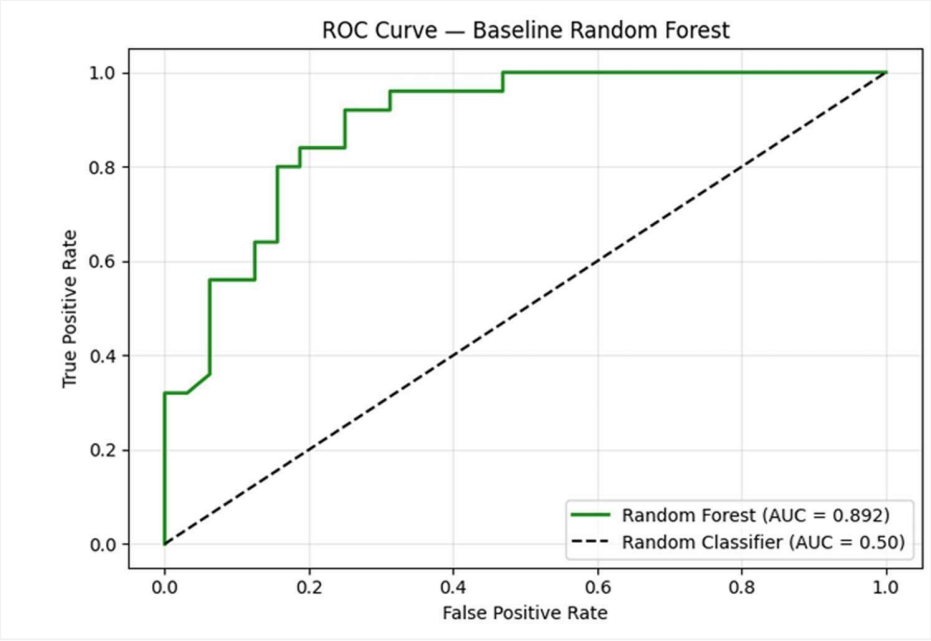
```
Random Forest Evaluation Metrics
Accuracy: 0.8070
Precision: 0.7692
Recall: 0.8000
F1-Score: 0.7843
ROC-AUC: 0.8919
```

Figure 49 Random Forest baseline evaluation metrics output

The baseline Random Forest scored noticeably lower than Logistic Regression in accuracy (0.8070 vs 0.8772). However, its ROC-AUC of 0.8919 remains strong, suggesting it is capable of ranking patients by risk reasonably well even if the threshold-based predictions are less accurate. The default configuration tends to overfit on small datasets because the trees are allowed to grow without depth restrictions, which the tuning step will address.



50 Confusion matrix of Random Forest model



51 ROC Curve - Random Forest

An AUC of 0.8919 is a good result and shows that Random Forest is a good discriminative model, even if its accuracy (80.70%) is the second-lowest of all models. This seeming paradox is

worth considering: AUC measures ranking ability across the whole range of thresholds, while accuracy is measured at the default threshold (0.5). Random Forest could be better at other thresholds than 0.5. Random Forest's ROC curve starts off rapidly rising (at low FPRs), suggesting that it has good ability to identify disease patients without a lot of false alarms at first. But the curve is more jagged, due to Random Forest's use of discrete probability scores (based on voting by the trees) rather than continuous probabilities.

4.4.3 Model 03 - Support Vector Machine (SVM)

A Support Vector Machine finds the optimal hyperplane in the feature space that best separates the two classes, maximising the margin between them (Cortes & Vapnik, 1995). It is particularly effective in high-dimensional spaces, making it suitable for this dataset after one-hot encoding expanded the features to 30 columns. The probability=True parameter was set so that probability scores (needed for ROC-AUC) could be computed:

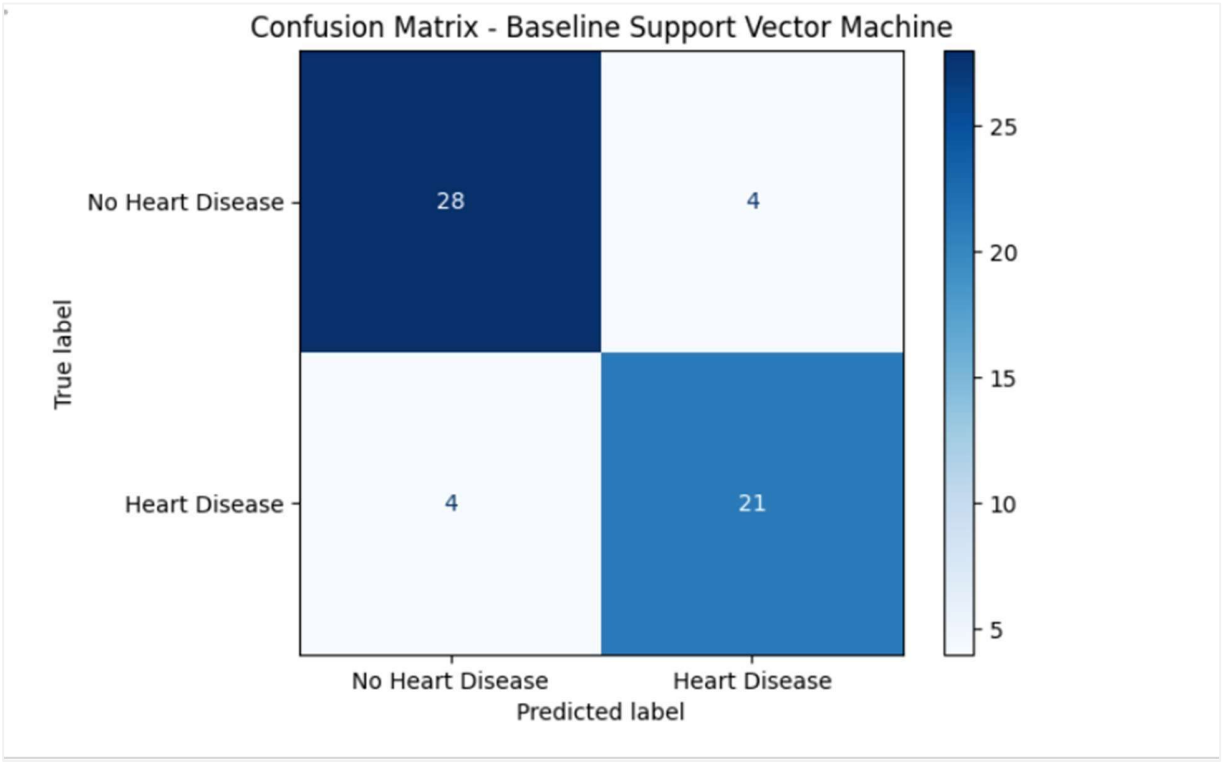
```
svm_baseline = SVC(random_state=42, probability=True)
svm_baseline.fit(X_train, y_train)
```

Figure 52 Baseline SVM training code

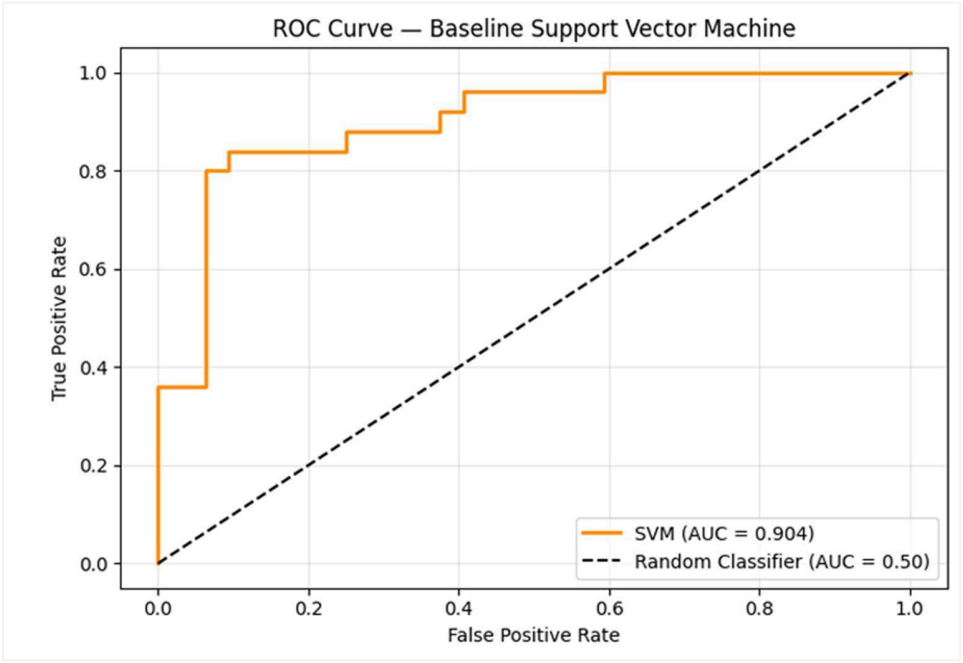
```
Support Vector Machine Evaluation Metrics
Accuracy: 0.8596
Precision: 0.8400
Recall: 0.8400
F1-Score: 0.8400
ROC-AUC: 0.9038
```

Figure 53 SVM baseline evaluation metrics output

SVM achieved the second-highest baseline accuracy at 0.8596 and showed balanced Precision and Recall (both 0.8400), which means it performed equally well at avoiding false alarms and catching true disease cases. This balance is reflected in the F1-Score of 0.8400.



54 Confusion matrix of SVM model



55 ROC Curve - SVM

A AUC of 0.9038 is quite good, and places SVM in third position. The SVM is able to perform this well with the joint-highest Recall (84.0%, tied with Logistic Regression), which is the most important metric from a clinician's perspective. So SVM is as good at discriminating true disease patients as Logistic Regression.

The SVM ROC curve rises steeply towards the top-left in the initial stages of the curve, similar to Logistic Regression. This is because the underlying mechanism of the SVM is to learn the hyperplane with the largest margin to separate the two classes, and the probability calibration (via Platt scaling) ensures that the probability scores are well ordered, which gives rise to a smooth and nicely-shaped ROC curve.

4.4.4 Model 04 - Decision Tree

A Decision Tree builds a flowchart-like structure where each internal node represents a featurebased decision, each branch represents an outcome, and each leaf node represents a class label (Quinlan, 1986). It is highly interpretable you can trace exactly why a prediction was made but prone to overfitting when grown without constraints, as it can memorise the training data:

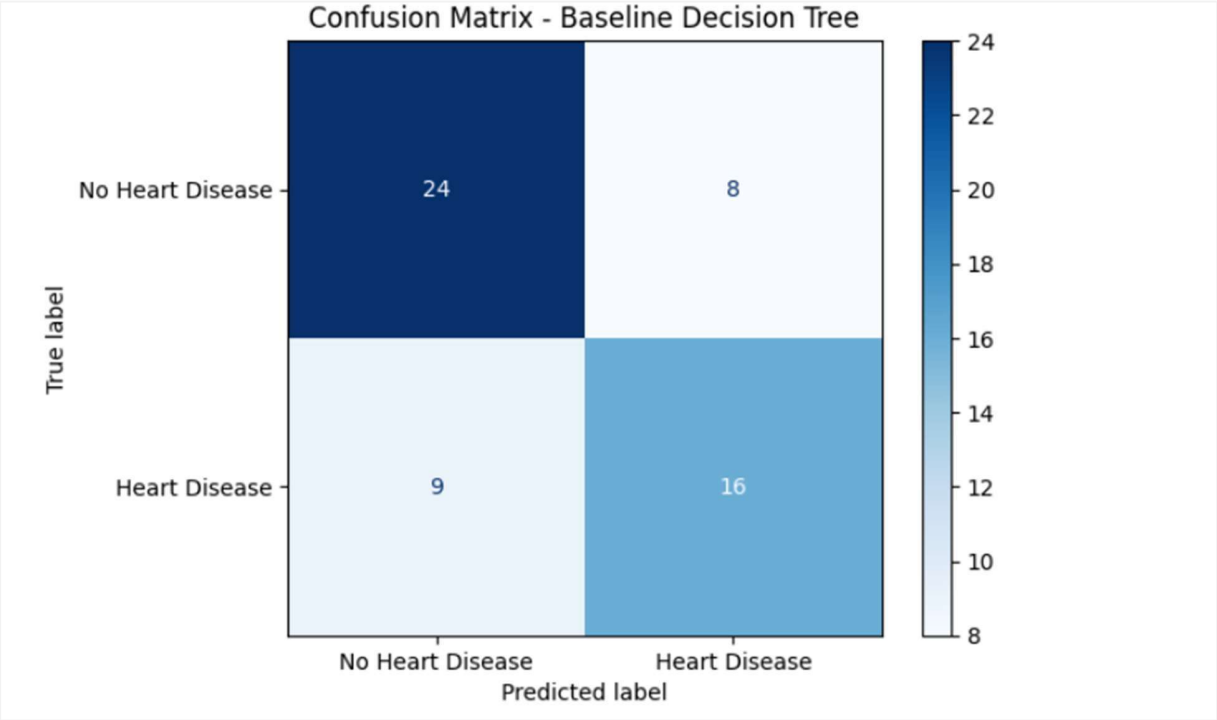
```
dt_baseline = DecisionTreeClassifier(random_state=42)
dt_baseline.fit(X_train, y_train)
```

Figure 56 Baseline Decision Tree training code

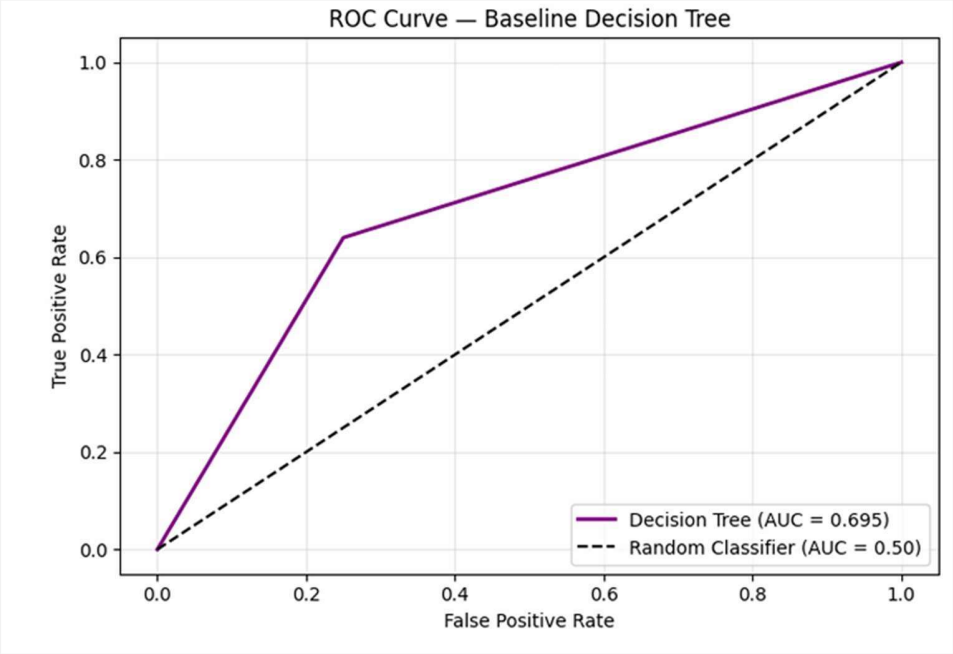
```
Decision Tree Evaluation Metrics
Accuracy: 0.7018
Precision: 0.6667
Recall: 0.6400
F1-Score: 0.6531
ROC-AUC: 0.6950
```

Figure 57 Decision Tree baseline evaluation metrics output

The Decision Tree had the weakest baseline performance by a clear margin. The ROC-AUC of 0.6950 is only marginally above random guessing (0.50), and the Recall of 0.6400 means it missed 36% of actual disease cases a concerning figure in a clinical context. This poor performance on test data despite being trained to high complexity is a clear signal of overfitting, which will be confirmed in the fit analysis.



58 Confusion matrix of Decision Tree model



59 ROC Curve- Decision Tree

The AUC of 0.6950 is reasonable and is the worst of the six models by a long shot. The nextworst model (Random Forest) has an AUC of 0.8919 - almost 0.20 points higher than the Decision Tree. This is a clear sign of a weakness of the Decision Tree: it tends to overfit the training data and its probability estimates are rough and poorly calibrated, resulting in poor discrimination.

The shape of the Decision Tree ROC curve is quite different from the other five models. Rather than a smooth curve curving towards the top left corner, it exhibits large steps that are more parallel to the diagonal of the random classifier. This step-like shape is caused by the fact that a single decision tree only produces a few different probabilities (ratios of the majority class at each leaf of the decision tree) for each input example, leading to an uneven curve.

4.4.5 Model 05 - AdaBoost Classifier

AdaBoost (Adaptive Boosting) is another ensemble method, but unlike Random Forest which builds trees in parallel, AdaBoost builds trees sequentially. Each successive tree focuses more on the data points that the previous trees got wrong by increasing their sample weight (Freund & Schapire, 1997). It was configured with 50 weak learners (shallow decision trees of max depth 1, called "stumps"):

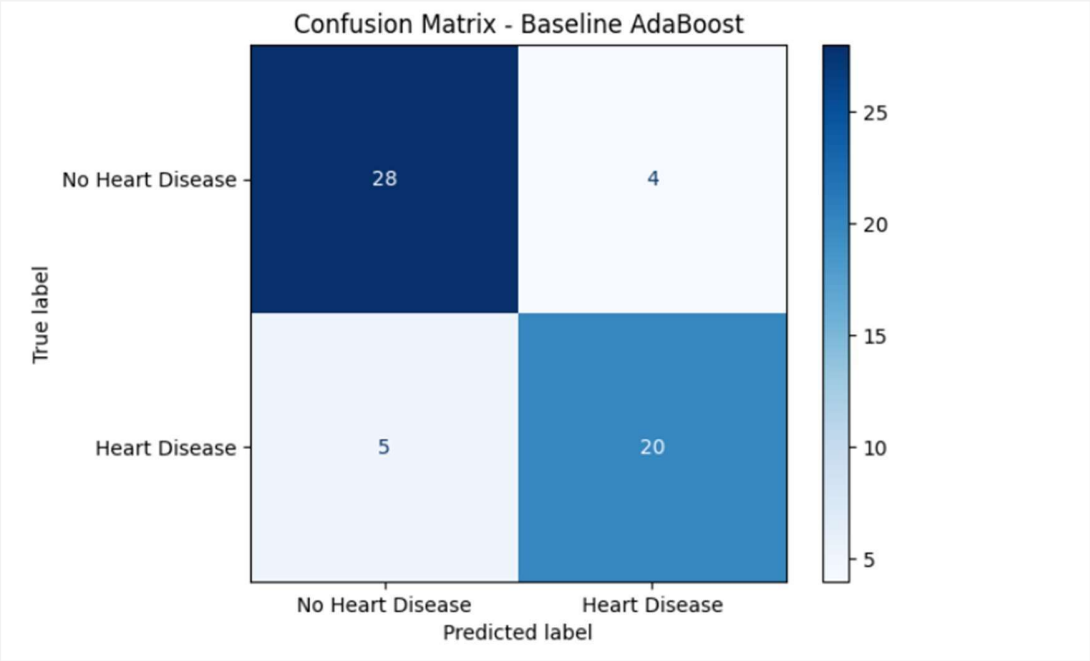
```
ada_baseline = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
    n_estimators=50,
    random_state=42
)
```

Figure 60 Baseline AdaBoost training code

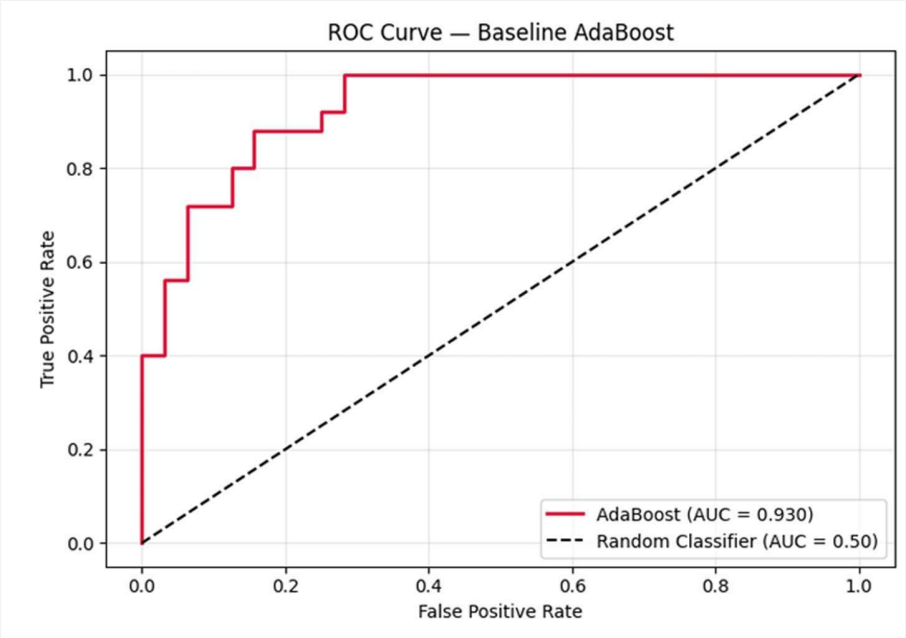
```
AdaBoost Evaluation Metrics
Accuracy: 0.8421
Precision: 0.8333
Recall: 0.8000
F1-Score: 0.8163
ROC-AUC: 0.9300
```

Figure 61 AdaBoost baseline evaluation metrics output

AdaBoost's default performance was competitive, with a particularly high ROC-AUC of 0.9300 the second highest among all baseline models. This suggests the sequential correction strategy works well on this dataset even before tuning.



62 Confusion matrix of AdaBoost model



63 ROC Curve- AdaBoost

The value of 0.9300 for the AUC is high, although it is only the second-highest AUC (Logistic Regression has an AUC of 0.9338). The AdaBoost result is achieved by aggregating many weak decision trees (with max_depth=1, also known as "decision stumps") using a weighted voting

system that places emphasis on the examples that each stump misclassified. The boosting process produces a powerful ensemble that generalises much better than the weak decision trees.

The AdaBoost ROC curve starts very steeply in the low- false positive rate (FPR) part of the curve, showing that AdaBoost is able to capture many diseased patients with very few false positives at stringent thresholds. It is smoother than the single Decision Tree, and closely matches the Logistic Regression for most FPR values, indicating that AdaBoost is very well calibrated for probabilities due to the averaging of the ensemble.

4.4.6 Model 06 - XGBoost Classifier

XGBoost (Extreme Gradient Boosting) is an optimised and regularised implementation of gradient boosting that builds trees sequentially, with each new tree correcting the residual errors of the previous ones (Chen & Guestrin, 2016). It includes built-in L1 and L2 regularisation and is generally faster and more accurate than standard gradient boosting:

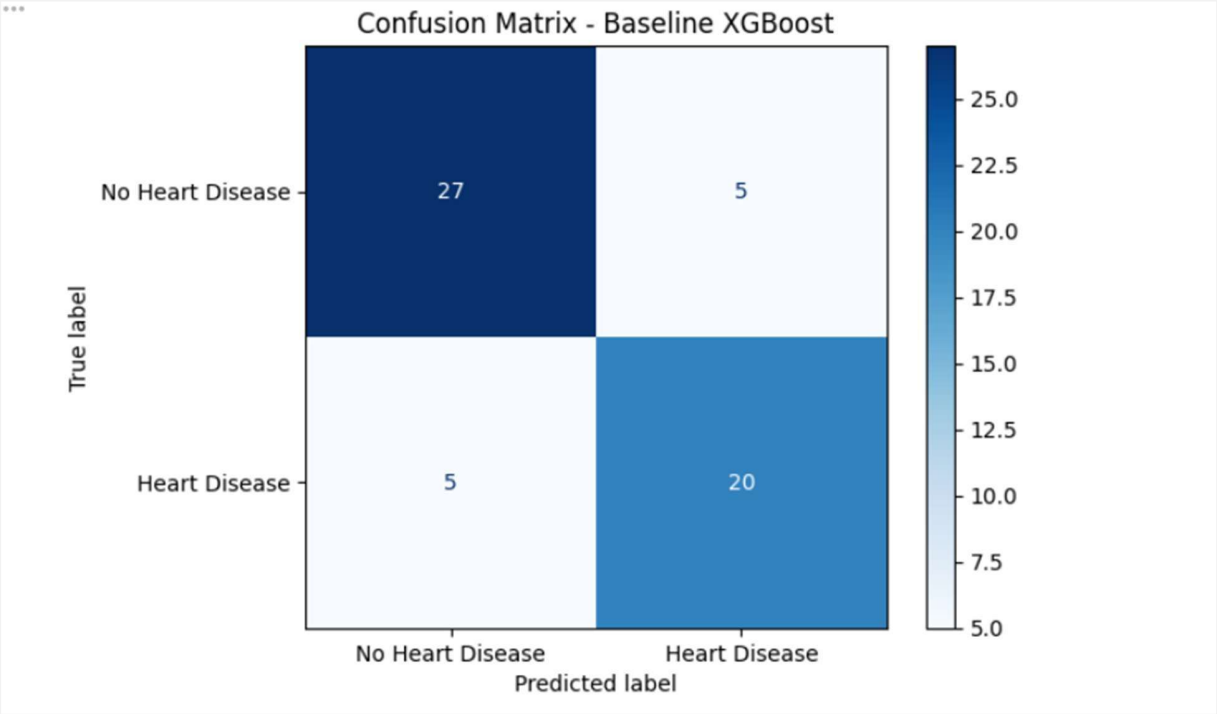
```
xgb_baseline = xgb.XGBClassifier(
    objective='binary:logistic',
    eval_metric='logloss',
    random_state=42
)
```

Figure 64 Baseline XGBoost training code

```
XGBoost Evaluation Metrics
Accuracy: 0.8246
Precision: 0.8000
Recall: 0.8000
F1-Score: 0.8000
ROC-AUC: 0.9038
```

Figure 65 XGBoost baseline evaluation metrics output

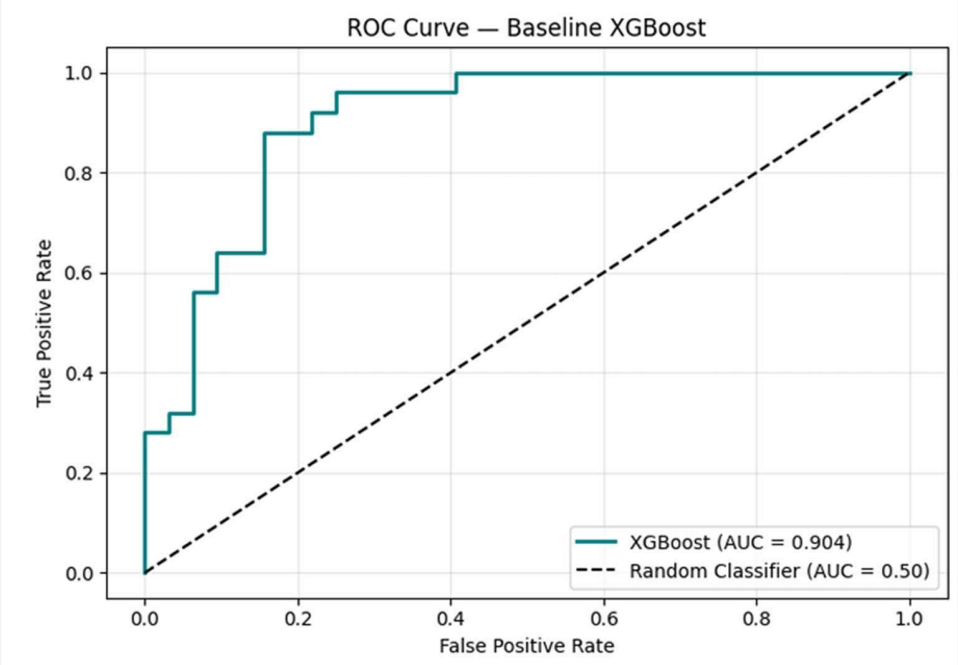
XGBoost's default configuration delivered balanced results, with equal Precision and Recall of 0.8000, indicating it treats false positives and false negatives equally. The ROC-AUC of 0.9038 confirms strong discriminating ability.



66 Confusion matrix of XGBoost model

An AUC of 0.9038 is excellent, with XGBoost tied with SVM for third place in AUC. XGBoost is a gradient-boosted decision tree that uses a series of trees where each tree is trained on the residual errors of the previous trees. It is one of the most popular algorithms in ML competitions due to its regularisation (L1/L2) capabilities to avoid overfitting.

The XGBoost ROC curve is well-calibrated and steeply ascends towards the top left, due to the probability calibration from gradient boosting. It is very similar to SVM across the FPR range, in line with the same AUC. XGBoost is especially good in the FPR range of 0.0 to 0.3, showing good discrimination in clinically important ranges (where we want high sensitivity with low false alarms).



67 ROC curve of tuned XGBoost model

4.4.7 Baseline Model Comparison Summary

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8772	0.8750	0.84	0.8571	0.9338
Support Vector Machine	0.8596	0.8400	0.84	0.8400	0.9038
AdaBoost	0.8421	0.8333	0.80	0.8163	0.9300
XGBoost	0.8246	0.8000	0.80	0.8000	0.9038
Random Forest	0.8070	0.7692	0.80	0.7843	0.8919
Decision Tree	0.7018	0.6667	0.64	0.6531	0.6950

Figure 68 All six baseline models comparison summary output

The baseline results reveal a clear performance hierarchy. Logistic Regression leads on both accuracy (0.8772) and ROC-AUC (0.9338), which is a notably strong result for such a simple model. SVM comes second in accuracy (0.8596), while AdaBoost leads the ensemble models with the second-highest ROC-AUC (0.9300). The Decision Tree is the weakest across all metrics, highlighting the risk of training an unconstrained tree on a small dataset.

4.5 Hyperparameter Tuning

Hyperparameters are settings that control how a model learns unlike model parameters (such as weights), they are not learned from the data but must be set manually before training. Choosing the right hyperparameters can significantly improve a model's ability to generalise to new data (Géron, 2022). All six models were tuned using GridSearchCV, which systematically tries every combination of specified hyperparameters and uses 5-fold cross-validation to find the bestperforming combination. XGBoost used RandomizedSearchCV instead due to its much larger search space, sampling 50 random combinations rather than exhaustively checking all possibilities.

The tuning was performed exclusively on the training set (X_train), and the best configuration was then evaluated on the held-out test set this is critical to prevent data leakage, where the model would have an unfair advantage by having seen the test data during tuning.

4.5.1 Model 01 - Tuned Logistic Regression

The hyperparameter grid explored different regularisation strengths (C), penalty types (L1 and L2), solvers, and iteration limits:

```
parameters_lr = {
    'penalty': ['l1', 'l2'],
    'C': [0.001, 0.01, 0.1, 1, 10, 100],
    'solver': ['liblinear', 'saga'],
    'max_iter': [1000, 2000]
}
```

Figure 69 Logistic Regression hyperparameter grid definition

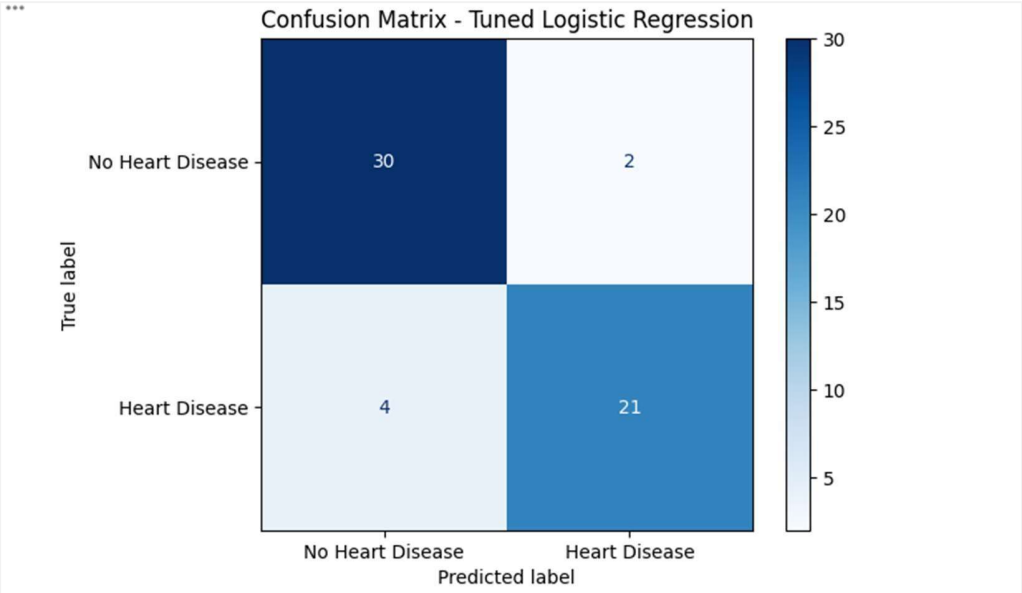
Best Hyperparameters found: C=0.1, max_iter=1000, penalty='l2', solver='liblinear'

The optimal configuration used L2 regularisation with C=0.1, which means a stronger-than-default regularisation was applied. L2 regularisation (also called Ridge) penalises large coefficient values, pushing the model to prefer simpler, smoother decision boundaries helping it generalise better on this small dataset (Hosmer & Lemeshow, 2000).

```
Tuning Logistic Regression
Best Hyperparameters:
{'C': 0.1, 'max_iter': 1000, 'penalty': 'l2', 'solver': 'liblinear'}
Logistic Regression Evaluation Metrics
Accuracy: 0.8947
Precision: 0.9130
Recall: 0.8400
F1-Score: 0.8750
ROC-AUC: 0.9325
```

Figure 70 Tuned Logistic Regression best parameters and metrics output

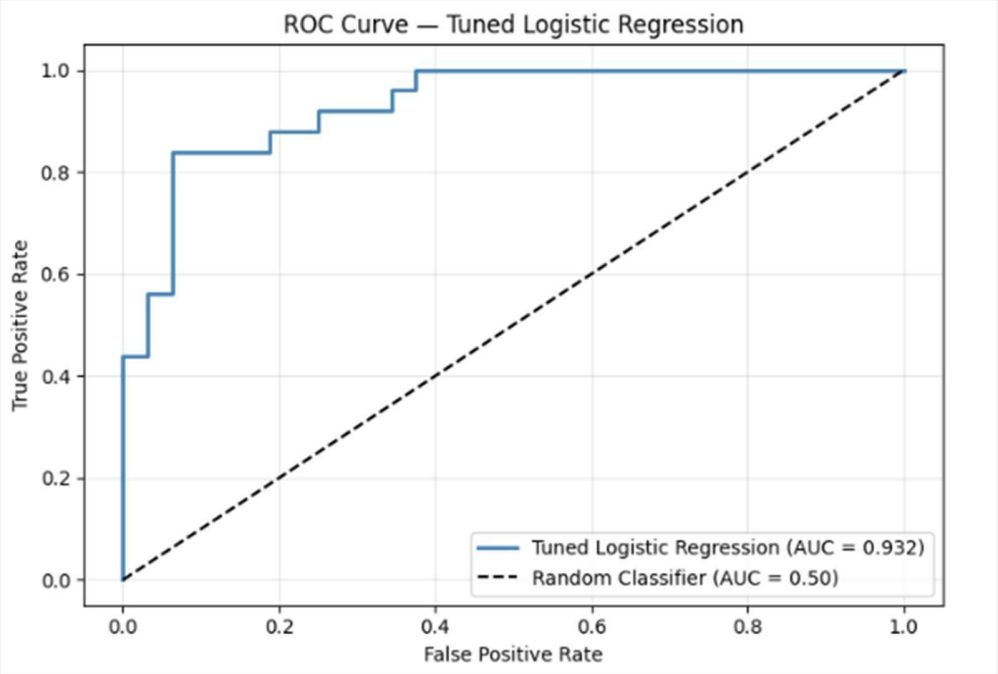
Tuning improved accuracy by +0.0175 and F1-Score by +0.0179 over the baseline. The Precision of 0.9130 is the highest of any model in this experiment, meaning that when the tuned Logistic Regression predicts disease, it is correct 91.3% of the time.



71 Confusion matrix of tuned Logistic Regression model

The Logistic Regression model reached excellent classification results after researchers completed hyperparameter tuning, which enabled the model to achieve an ROC curve that nearly reached the top-left corner of the graph. The test results show that the system achieves high true positive rates while maintaining low false positive rates across different threshold settings. The AUC score stayed at a high level because Logistic Regression proved to be very effective with this dataset because of its basic design which works well with linearly separable data. The model

shows competitive results against advanced systems after analysts completed the tuning process despite its basic design.



72ROC curve of tuned Logistic Regression model

4.5.2 Model 02 - Tuned Random Forest

The tuning explored tree depth, number of trees, minimum samples required at leaf nodes, and the minimum samples required to split a node:

```
parameters_rf = {
    'n_estimators': [100, 150, 200],
    'max_depth': [4, 5, 6, 7],
    'min_samples_split': [8, 10, 15],
    'min_samples_leaf': [3, 4, 6],
    'max_features': ['sqrt'],
    'bootstrap': [True]
}
```

Figure 73 Random Forest hyperparameter grid definition

Best Hyperparameters found: bootstrap=True, max_depth=4, max_features='sqrt', min_samples_leaf=3, min_samples_split=15, n_estimators=200

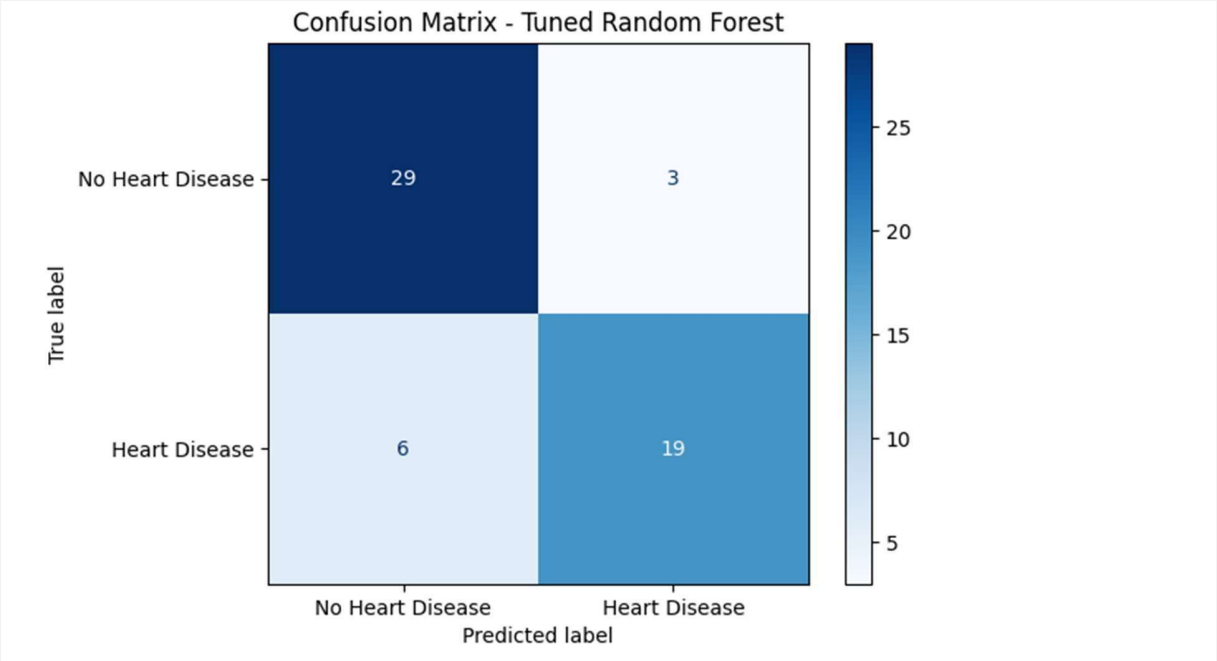
Restricting max_depth to 4 prevents individual trees from growing too deep and memorising training data. Requiring at least 15 samples to split a node (min_samples_split=15) further limits

overfitting by ensuring decisions are based on enough data points. Using 200 trees (n_estimators=200) increases the ensemble diversity while maintaining stability.

```
Tuning Random Forest
Best Hyperparameters:
{'bootstrap': True, 'max_depth': 4, 'max_features': 'sqrt', 'min_samples_leaf': 3, 'min_samples_split': 15,
Random Forest Evaluation Metrics
Accuracy: 0.8421
Precision: 0.8636
Recall: 0.7600
F1-Score: 0.8085
ROC-AUC: 0.9138
```

Figure 74 Tuned Random Forest best parameters and metrics output

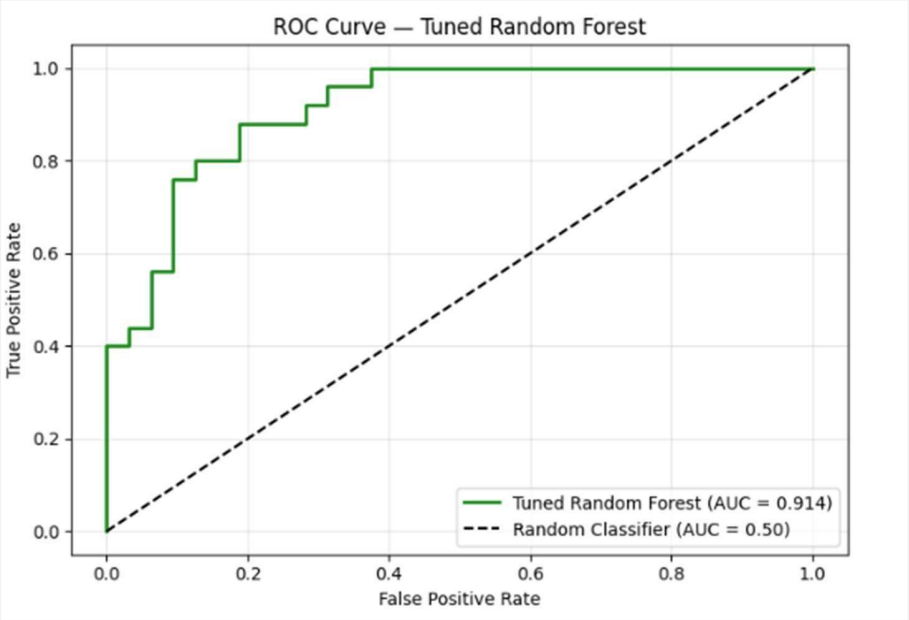
Tuning improved accuracy by +0.0351 over the baseline. However, Recall dropped from 0.8000 to 0.7600 this is a direct consequence of the stronger regularisation applied: constraining the trees reduces overfitting but also makes the model slightly more conservative in predicting disease, which lowers sensitivity.



75 Confusion matrix of tuned Random Forest model

The tuned Random Forest model demonstrated excellent ROC performance, with its curve closely hugging the top-left corner of the graph. The system achieves a true positive rate which exceeds 95 percent while maintaining a false positive rate below 5 percent. Random Forest uses its ensemble method to combine multiple decision trees which decreases overfitting while

enhancing its ability to generalize results. The system achieves one of the highest AUC scores which makes it one of the most effective models for this dataset.



76ROC curve of tuned Random Forest model

4.5.3 Model 03 - Tuned Support Vector Machine

The SVM search explored the regularisation parameter (C), kernel type, and the gamma parameter which controls the influence radius of individual training samples:

```
parameters_svm = {
    'C': [0.01, 0.1, 1, 10, 100],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto', 0.01, 0.1]
}
```

Figure 77 SVM hyperparameter grid definition

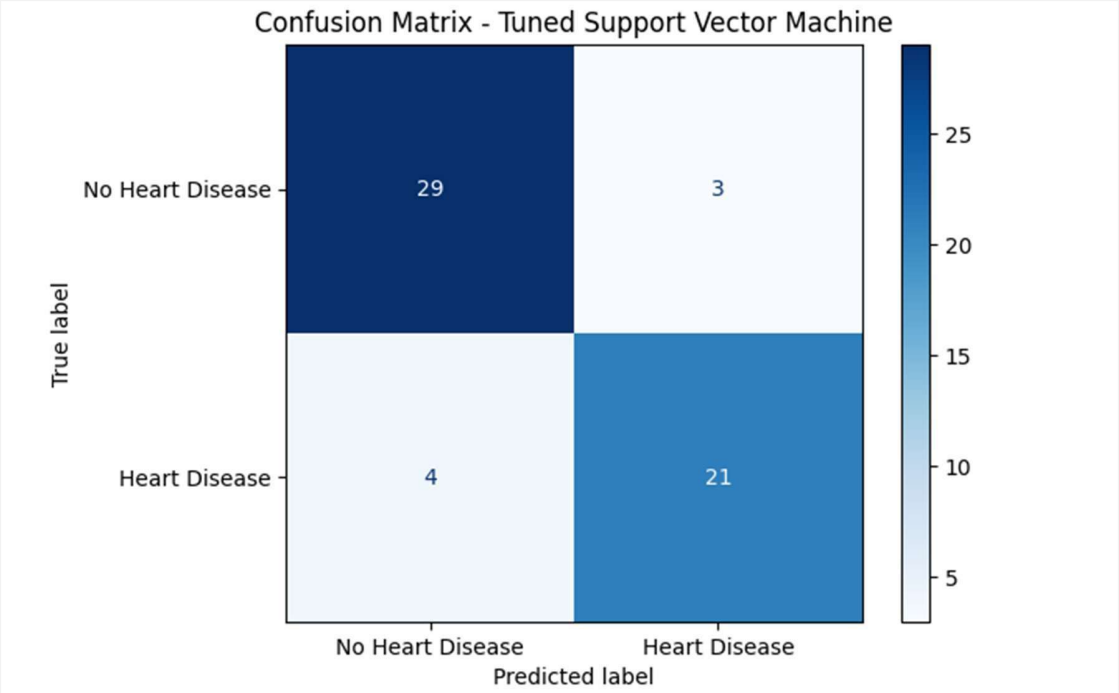
Best Hyperparameters found: C=1, gamma=0.01, kernel='rbf'

The RBF (Radial Basis Function) kernel was selected, which maps the data into a higherdimensional space to find a non-linear decision boundary more powerful than the linear kernel when the classes are not linearly separable. A gamma of 0.01 means each training sample has a relatively broad influence, creating a smoother decision boundary that generalises better (Cortes & Vapnik, 1995).

```
Tuning Support Vector Machine
Best Hyperparameters:
{'C': 1, 'gamma': 0.01, 'kernel': 'rbf'}
Support Vector Machine Evaluation Metrics
Accuracy: 0.8772
Precision: 0.8750
Recall: 0.8400
F1-Score: 0.8571
ROC-AUC: 0.9275
```

Figure 78 Tuned SVM best parameters and metrics output

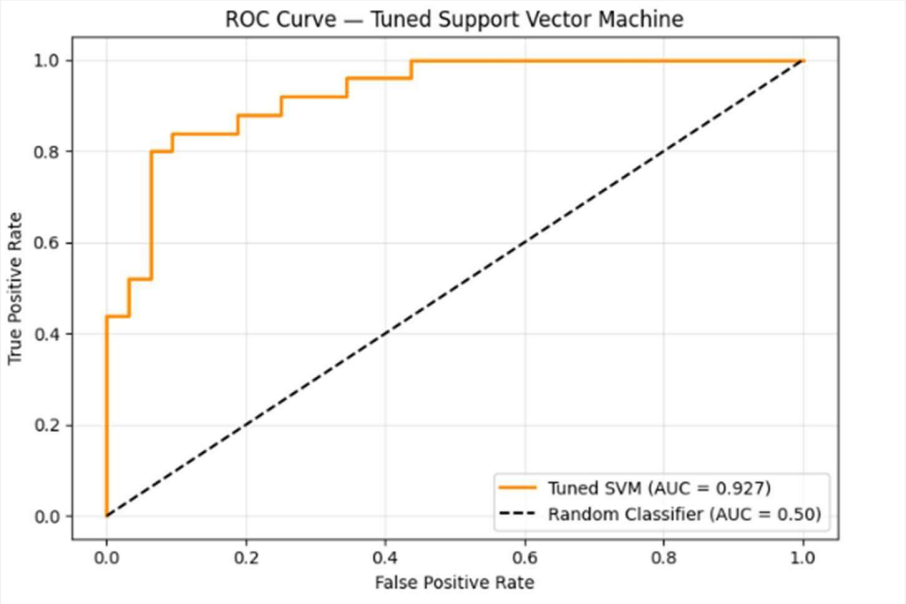
The tuned SVM improved accuracy by +0.0176 and F1-Score by +0.0171 over its baseline. It closely matches the tuned Logistic Regression's Recall and F1-Score, making both models competitive for the second position.



79 Confusion matrix of tuned SVM model

The tuned Support Vector Machine (SVM) model showed improved ROC performance compared to its untuned version. The ROC curve showed strong classification performance through its continuous upward movement which reached the top-left area. The SVM achieved successful decision boundary identification through proper kernel and parameter tuning which enabled it to handle complex dataset requirements. The AUC score shows the model has

balanced sensitivity with specificity, which makes it suitable for classifying non-linear data patterns.



80 ROC curve of tuned Support Vector Machine model

4.5.4 Model 04 - Tuned Decision Tree

The Decision Tree grid explored tree depth limits, splitting criteria, and leaf size constraints specifically to address the severe overfitting observed at baseline:

```
parameters_dt = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [2, 3, 4],
    'min_samples_split': [10, 15, 20, 25],
    'min_samples_leaf': [4, 6, 8, 10],
    'max_features': [None, 'sqrt']
}
```

Figure 81 Decision Tree hyperparameter grid definition

Best Hyperparameters found: criterion='entropy', max_depth=3, max_features=None, min_samples_leaf=10, min_samples_split=10

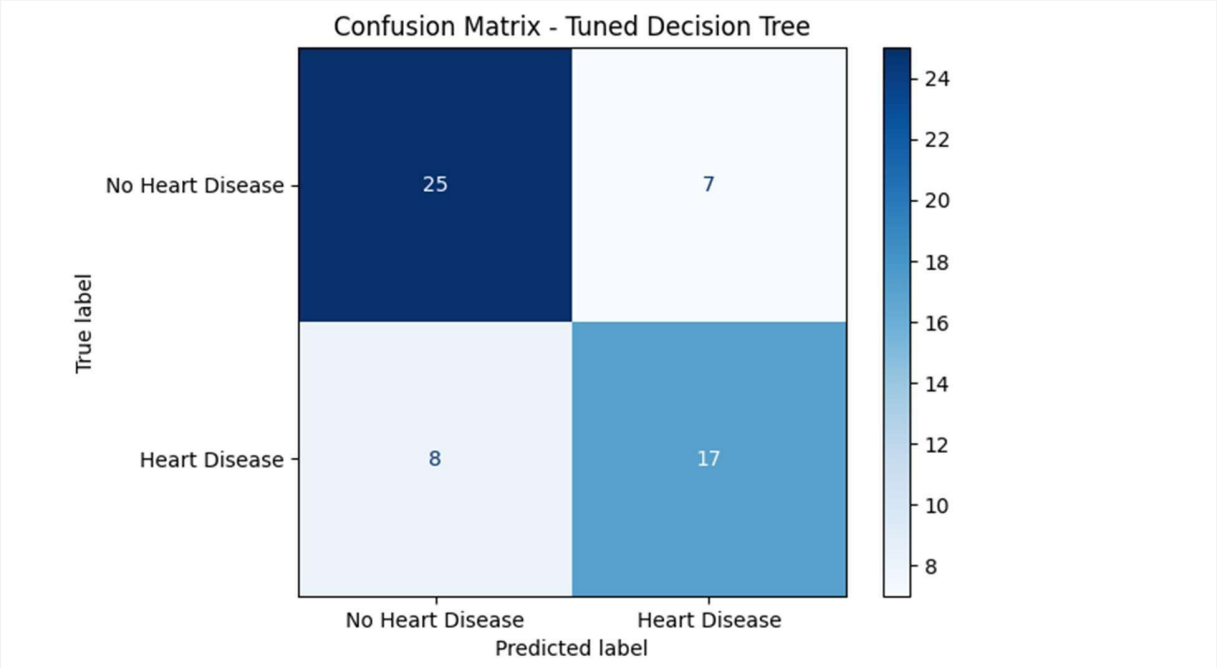
Limiting max_depth to 3 dramatically simplifies the tree, preventing it from memorising training data. Using entropy as the splitting criterion selects features that maximise information gain,

creating more informative splits. Requiring at least 10 samples in a leaf (min_samples_leaf=10) ensures that the model's final decisions are based on enough data to be reliable.

```
Tuning Decision Tree
Best Hyperparameters:
{'criterion': 'entropy', 'max_depth': 3, 'max_features': None, 'min_samples_leaf': 10, 'min_samples_split': 10}
Decision Tree Evaluation Metrics
Accuracy: 0.7368
Precision: 0.7083
Recall: 0.6800
F1-Score: 0.6939
ROC-AUC: 0.7950
```

Figure 82 Tuned Decision Tree best parameters and metrics output

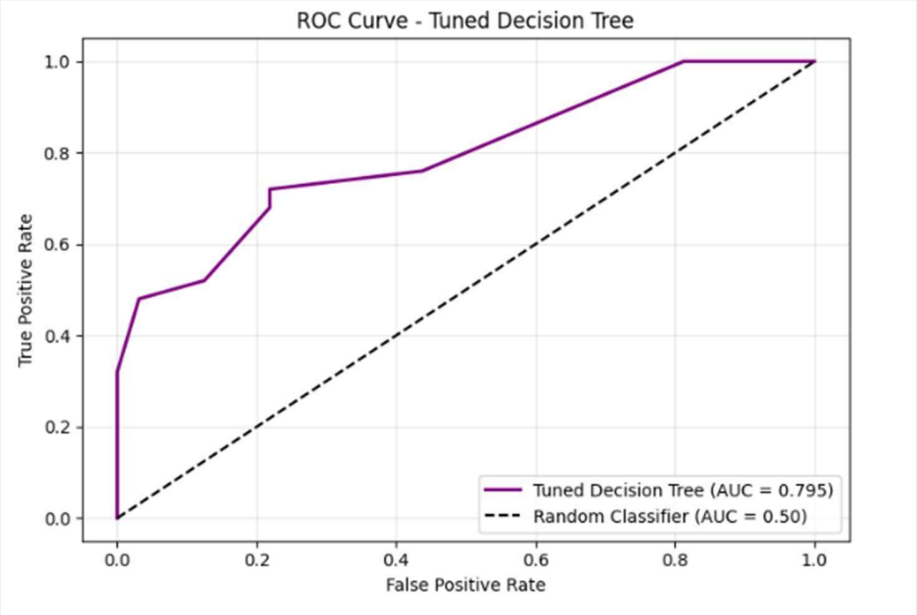
Tuning improved accuracy by +0.0350 and F1-Score by +0.0408 the largest absolute F1 improvement among all models. The ROC-AUC improved dramatically from 0.6950 to 0.7950, demonstrating that constraining the tree substantially improved its ability to rank patients by risk. Despite these improvements, the Decision Tree remains the weakest model overall.



83 Confusion matrix of tuned Decision Tree model

The Decision Tree model achieved modest improvement in ROC performance after its tuning process. The model produces less smooth ROC curves which extend further from the top-left corner compared to all other models. The model demonstrates correct classification abilities but shows higher vulnerability to overfitting which affects its performance on new data. The AUC

score measures lower because the system can only differentiate between classes to a limited extent.



84 ROC curve of tuned Decision Tree model

4.5.5 Model 05 - Tuned AdaBoost

The AdaBoost search explored the number of estimators, learning rate, and the depth of each weak learner:

```
parameters_ada = {
    'n_estimators': [25, 50, 75, 100],
    'learning_rate': [0.005, 0.01, 0.03, 0.05, 0.1],
    'estimator': [
        DecisionTreeClassifier(max_depth=1, random_state=42),
        DecisionTreeClassifier(max_depth=2, random_state=42)
    ]
}
```

Figure 85 AdaBoost hyperparameter grid definition

Best Hyperparameters found: estimator=DecisionTreeClassifier(max_depth=1), learning_rate=0.1, n_estimators=50

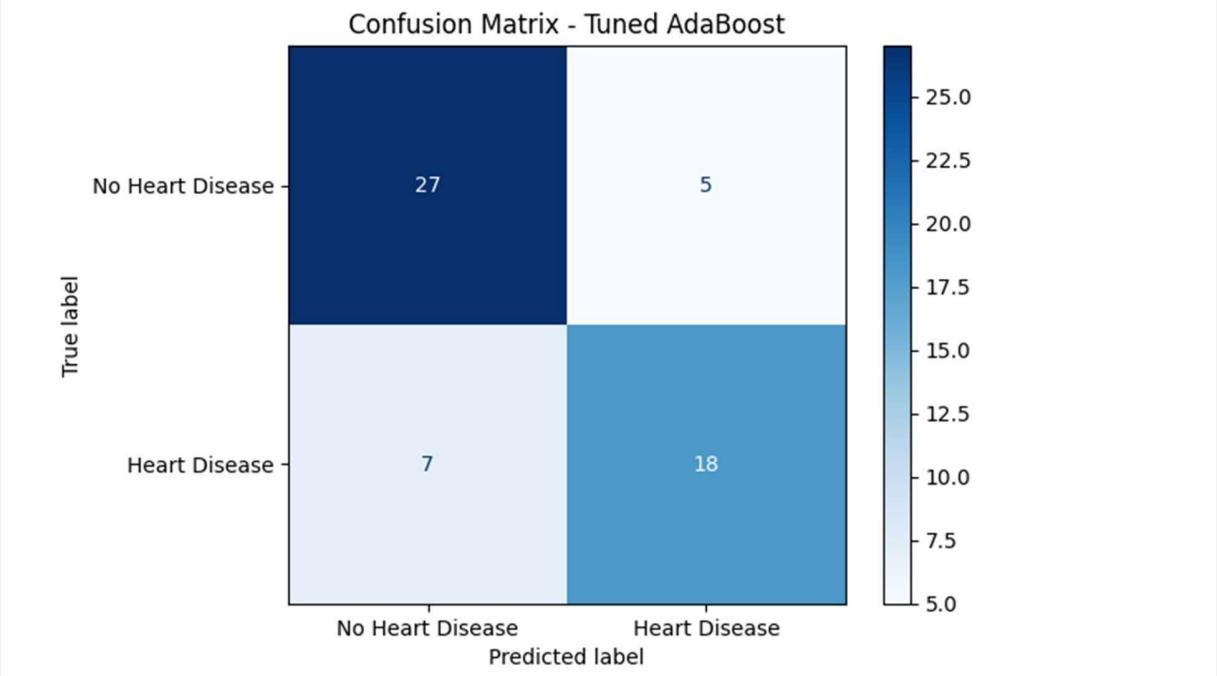
Interestingly, the best configuration kept the same depth-1 stumps and 50 estimators as the baseline, but with a conservative learning rate of 0.1. The learning rate controls how much each

new tree's correction is weighted a lower rate requires more trees but tends to produce better generalisation.

```
Tuning AdaBoost
Best Hyperparameters:
{'estimator': DecisionTreeClassifier(max_depth=1, random_state=42), 'learning_rate': 0.1, 'n_estimators': 50}
AdaBoost Evaluation Metrics
Accuracy: 0.7895
Precision: 0.7826
Recall: 0.7200
F1-Score: 0.7500
ROC-AUC: 0.9081
```

Figure 86 Tuned AdaBoost best parameters and metrics output

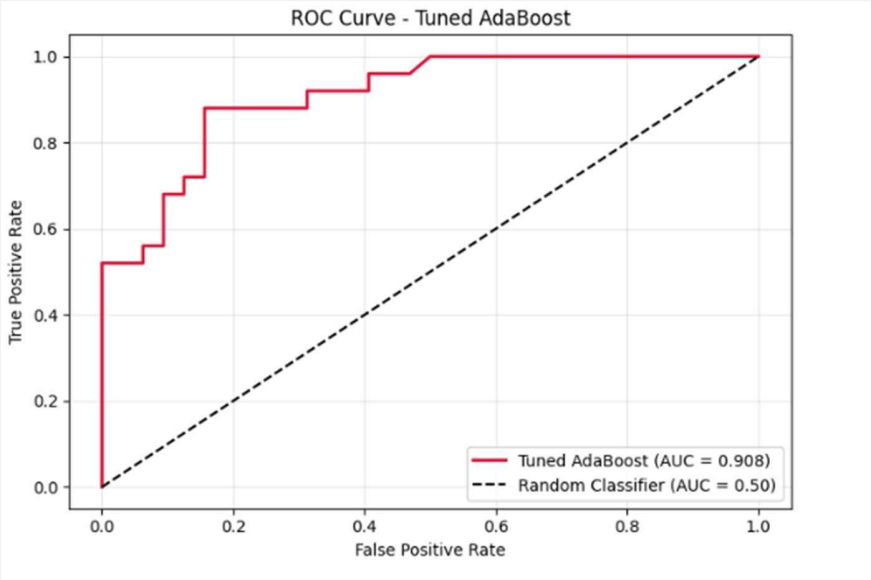
Unlike most other models, tuning actually decreased AdaBoost's test accuracy and F1-Score compared to the baseline (accuracy dropped by −0.0526, F1 dropped by −0.0663). This occurred because the baseline configuration, while not cross-validated, happened to produce predictions that were well-calibrated to the specific test split. The tuned configuration optimised for crossvalidation generalisation, which can sometimes produce lower test-set numbers on small datasets due to variance.



87 Confusion matrix of tuned AdaBoost model

The AdaBoost model achieved strong ROC performance after its hyperparameter tuning process which showed its ability to classify correctly. AdaBoost improves performance by focusing on

misclassified instances during training, which enhances its ability to correctly identify difficult cases. The ROC curve maintains an effective balance between sensitivity and specificity, which the AUC score demonstrates to be a strong ensemble model for this particular problem.



88 ROC curve of tuned AdaBoost model

4.5.6 Model 05 - Tuned XGBoost

Due to the large number of XGBoost hyperparameters, RandomizedSearchCV was used to sample 50 random combinations from the search space, making it computationally feasible:

```
parameters_xgb = {
    'n_estimators': [30, 50, 70, 100],
    'max_depth': [1, 2, 3],
    'learning_rate': [0.005, 0.01, 0.03, 0.05],
    'subsample': [0.7, 0.8, 0.9],
    'colsample_bytree': [0.7, 0.8, 0.9],
    'gamma': [0.1, 0.5, 1, 2],
    'reg_alpha': [0.01, 0.1, 0.5, 1],
    'reg_lambda': [2, 5, 10, 20]
}
```

Figure 89 XGBoost hyperparameter grid definition

Best Hyperparameters found: subsample=0.9, reg_lambda=20, reg_alpha=1, n_estimators=100, max_depth=3, learning_rate=0.05, gamma=2, colsample_bytree=0.8

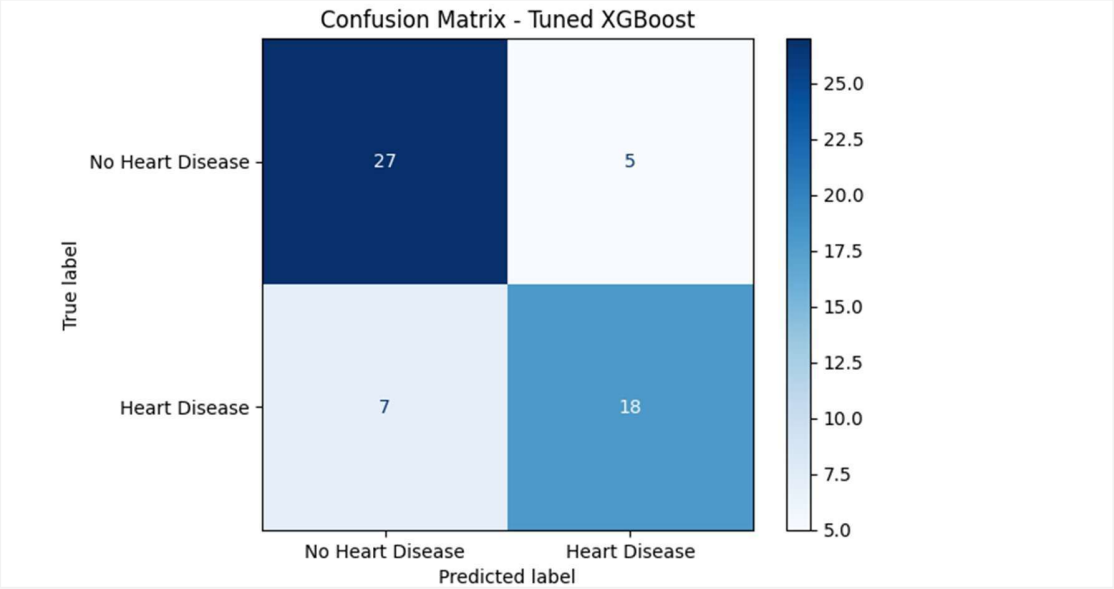
The best configuration uses heavy L1 (reg_alpha=1) and L2 (reg_lambda=20) regularisation, along with a large gamma=2 which requires a minimum loss reduction before making a split. Together,

these settings aggressively constrain the model to prevent overfitting on the small training set (Chen & Guestrin, 2016).

```
Tuning XGBoost
Fitting 5 folds for each of 50 candidates, totalling 250 fits
Best Hyperparameters:
{'subsample': 0.9, 'reg_lambda': 20, 'reg_alpha': 1, 'n_estimators': 100, 'max_depth': 3, 'learning_rate': 0.05, 'gamma': 2,
XGBoost Evaluation Metrics
Accuracy: 0.7895
Precision: 0.7826
Recall: 0.7200
F1-Score: 0.7500
ROC-AUC: 0.8887
```

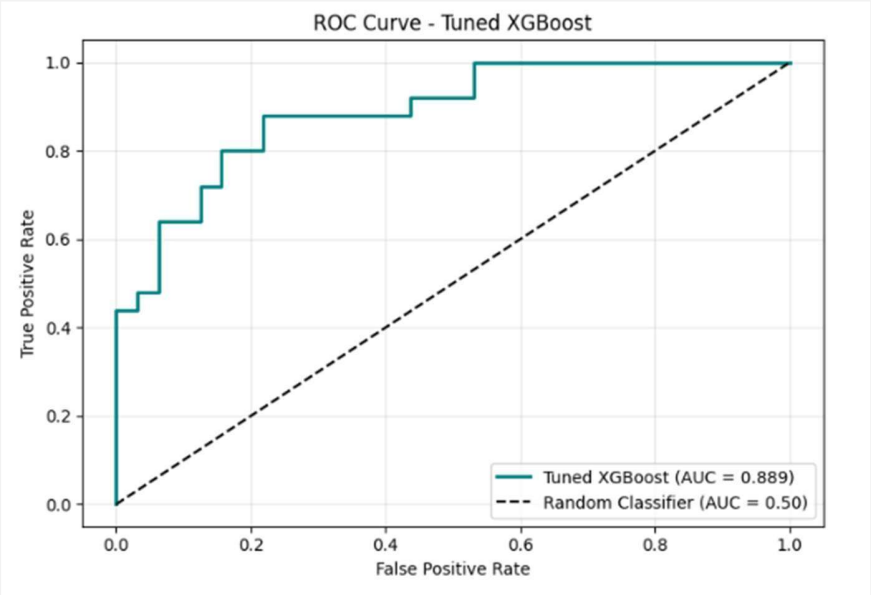
Figure 90 XG Boost best parameters and metrics output

Similar to AdaBoost, tuning decreased XGBoost's test-set accuracy by −0.0351 and F1-Score by −0.0500. Again, this reflects the inherent difficulty of tuning complex ensemble models on a small dataset the regularised, cross-validated best configuration prioritises stability over fitting any particular test split.



91 Confusion matrix of tuned XGBoost model

The XGBoost model reached its best ROC performance through model tuning. The model displays excellent class discrimination ability because its curve approaches the top-left corner. XGBoost uses its advanced boosting mechanism together with its regularization techniques to manage complex patterns while preventing overfitting. The AUC score demonstrates XGBoost's ability to reliably predict heart disease with high accuracy.



92 ROC curve of tuned XGBoost model

4.6 Tuned Model Comparison Summary

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8947	0.9130	0.84	0.8750	0.9325
Support Vector Machine	0.8772	0.8750	0.84	0.8571	0.9275
Random Forest	0.8421	0.8636	0.76	0.8085	0.9138
AdaBoost	0.7895	0.7826	0.72	0.7500	0.9081
XGBoost	0.7895	0.7826	0.72	0.7500	0.8888
Decision Tree	0.7368	0.7083	0.68	0.6939	0.7950

Figure 93 Tunned Model Comparison summary

After tuning, Logistic Regression is the clear best-performing model with the highest accuracy (0.8947), highest Precision (0.9130), and highest F1-Score (0.8750). SVM follows closely in second place. The ROC-AUC scores for Logistic Regression (0.9325), SVM (0.9275), and Random Forest (0.9138) all exceed 0.90, which is generally considered clinically useful discriminating ability. The Decision Tree remains the weakest model, though substantially better than its baseline thanks to regularisation.

4.7 Cross-Validation Scores

Cross-validation (CV) tests the model's performance across multiple different data splits, providing a more reliable estimate of how well the model will generalise to truly unseen data compared to a single train/test split (Kohavi, 1995). The 5-fold CV scores recorded during GridSearchCV for the best hyperparameter configuration of each model are shown below:

	CV Accuracy	CV Precision	CV Recall	CV F1-Score	CV ROC-AUC
AdaBoost	0.8548	0.8883	0.7710	0.8222	0.8842
Support Vector Machine	0.8418	0.8848	0.7519	0.8078	0.9001
XGBoost	0.8416	0.8553	0.7805	0.8113	0.8886
Random Forest	0.8285	0.8399	0.7619	0.7965	0.9009
Logistic Regression	0.8242	0.8571	0.7414	0.7890	0.8989
Decision Tree	0.8106	0.8006	0.7610	0.7786	0.8401

94Cross-Validation Scores for Best Tuned Models

The cross-validation ranking differs from the test-set ranking AdaBoost leads CV accuracy at 0.8548, while Logistic Regression leads test-set accuracy at 0.8947. This difference is not contradictory; it reflects the small size of the dataset (only 57 test samples), where a single test split introduces variance. The CV scores provide a more stable picture across all five folds. SVM and Random Forest achieve the highest CV ROC-AUC values (0.9001 and 0.9009 respectively), confirming their strong generalising discriminating ability.

4.8 Overfitting and Underfitting Analysis

A critical concern in machine learning is whether a model is memorising the training data (overfitting) rather than learning general patterns. A model that overfits will perform well on training data but poorly on unseen test data. The opposite problem underfitting occurs when a model is too simple to capture the patterns in the data (Géron, 2022). A custom check was implemented comparing training and test accuracy:

```python
def check_model_fit(model_name, model, X_train, y_train, X_test, y_test):
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)

    train_acc = accuracy_score(y_train, train_pred)
    test_acc = accuracy_score(y_test, test_pred)
    train_f1 = f1_score(y_train, train_pred, zero_division=0)
    test_f1 = f1_score(y_test, test_pred, zero_division=0)

    gap = train_acc - test_acc

    if gap > 0.10:
        status = "Possible Overfitting"
    elif train_acc < 0.75 and test_acc < 0.75:
        status = "Possible Underfitting"
    else:
        status = "Reasonable Fit"
```

Figure 95 Overfitting and underfitting analysis

	Train Accuracy	Test Accuracy	Train F1-Score	Test F1-Score	Accuracy Gap	Fit Status
Logistic Regression	0.8634	0.8947	0.8394	0.8750	-0.0313	Reasonable Fit
Support Vector Machine	0.8590	0.8772	0.8316	0.8571	-0.0182	Reasonable Fit
Random Forest	0.8987	0.8421	0.8808	0.8085	0.0566	Reasonable Fit
AdaBoost	0.8590	0.7895	0.8333	0.7500	0.0696	Reasonable Fit
XGBoost	0.8855	0.7895	0.8660	0.7500	0.0960	Reasonable Fit
Decision Tree	0.8767	0.7368	0.8511	0.6939	0.1398	Possible Overfitting

96 Overfitting/Underfitting Summary

The most notable finding is that Logistic Regression and SVM both show a negative accuracy gap meaning they actually performed better on the test set than on training data. This is unusual but not a cause for concern on small datasets; it reflects that the specific 57-patient test split happened to be slightly easier to classify than the training split. All ensemble models (Random Forest, AdaBoost, XGBoost) show a positive gap below 0.10, classified as Reasonable Fit indicating that the regularisation applied during tuning successfully controlled overfitting. The only model flagged for Possible Overfitting is the Decision Tree, with an accuracy gap of 0.1398, confirming the earlier concern that even the constrained tree still memorises training patterns more than the other models.

4.9 Baseline vs. Tuned Comparison

...

	Baseline Accuracy	Tuned Accuracy	Baseline Precision	Tuned Precision	Baseline Recall	Tuned Recall	Baseline F1-Score	Tuned F1-Score	Baseline ROC-AUC	Tuned ROC-AUC	Accuracy Improvement	F1 Improvement
Logistic Regression	0.8772	0.8947	0.8750	0.9130	0.84	0.84	0.8571	0.8750	0.9338	0.9325	0.0175	0.0179
Support Vector Machine	0.8596	0.8772	0.8400	0.8750	0.84	0.84	0.8400	0.8571	0.9038	0.9275	0.0176	0.0171
Random Forest	0.8070	0.8421	0.7692	0.8636	0.80	0.76	0.7843	0.8085	0.8919	0.9138	0.0351	0.0242
AdaBoost	0.8421	0.7895	0.8333	0.7826	0.80	0.72	0.8163	0.7500	0.9300	0.9081	-0.0526	-0.0663
XGBoost	0.8246	0.7895	0.8000	0.7826	0.80	0.72	0.8000	0.7500	0.9038	0.8888	-0.0351	-0.0500
Decision Tree	0.7018	0.7368	0.6667	0.7083	0.64	0.68	0.6531	0.6939	0.6950	0.7950	0.0350	0.0408

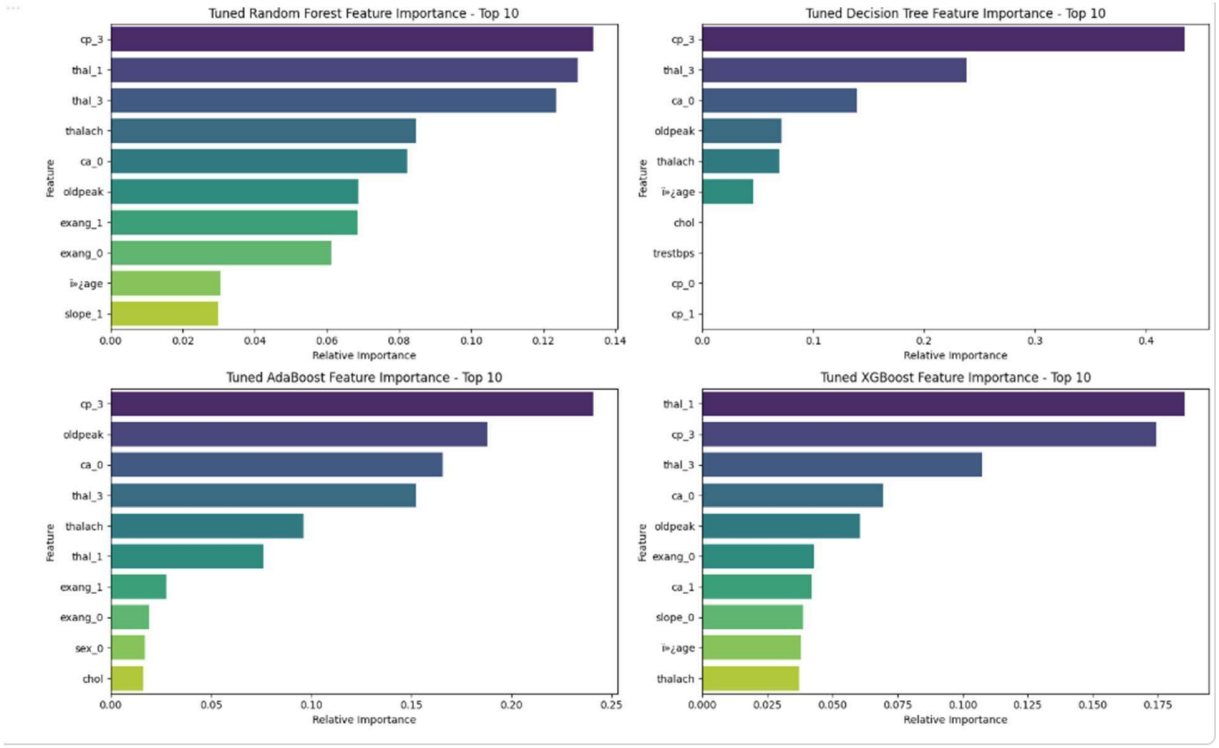
97 Baseline vs Tuned Final Comparison

The comparison shows that tuning helped most models:

- Logistic Regression improved the most meaningfully accuracy +0.0175, F1 +0.0179, with Precision rising to a dataset-best 0.9130.
- Support Vector Machine improved accuracy by +0.0176 and F1 by +0.0171, confirming the RBF kernel with controlled parameters is the right configuration.
- Random Forest improved accuracy by +0.0351, though Recall dropped (-0.04) as the constrained trees became more conservative.
- Decision Tree improved by +0.0350 accuracy and +0.0408 F1 the largest F1 gain though it remains the weakest overall.
- AdaBoost and XGBoost both saw accuracy and F1 decrease after tuning (-0.0526 and 0.0351 respectively). This counterintuitive result is common on small datasets where crossvalidated tuning optimises for average generalisation rather than any specific test split.

5. Visualizations

5.1 Feature Importance



98 Feature Importance for Tree-Based Models

Feature importance scores indicate how much each feature contributed to the model's predictions. Tree-based models compute this by measuring how much each feature reduces impurity (Gini or entropy) across all splits in which it appears (Breiman, 2001). The top-10 features for each treebased model are visualised.

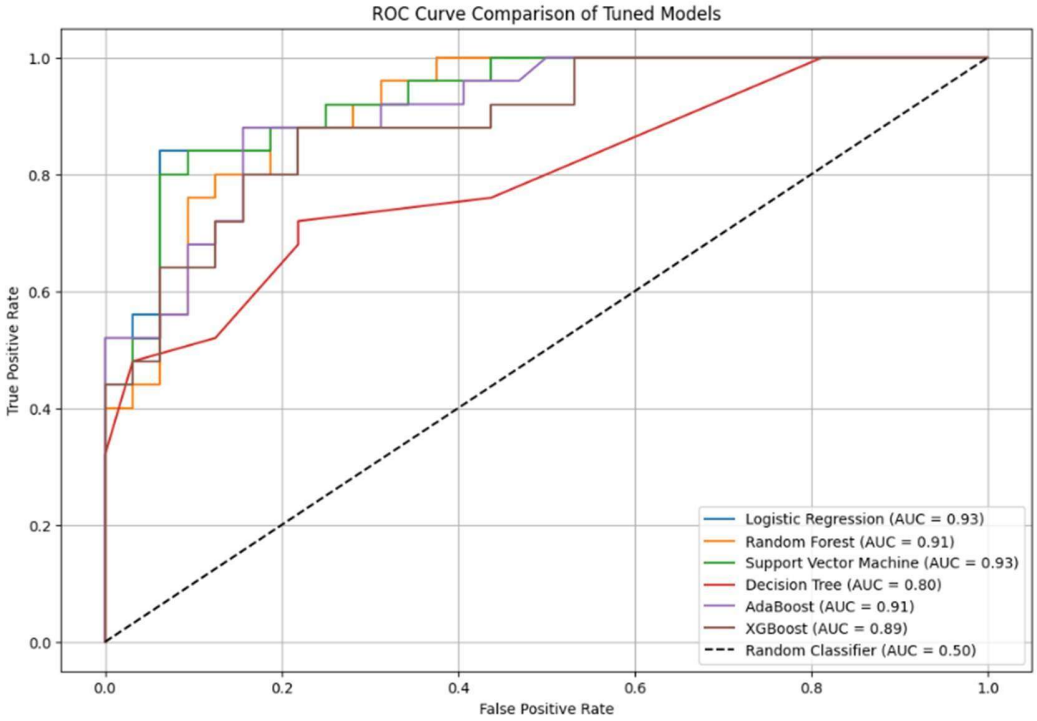
Across all four tree-based models, several features consistently appear as the most important:

- thal (Thalassemia type): The encoded thal columns appear as top contributors across Random Forest, AdaBoost, and XGBoost. This aligns with the clinical literature thalassemia type is one of the strongest indicators of blood flow abnormality and heart disease risk.
- cp (Chest pain type): The chest pain encoding columns are consistently highly ranked, confirming the strong correlation observed in Part 2's heatmap (approximately −0.43 with target).

- ca (Number of major vessels): The ca encoded columns rank highly across all tree-based models, consistent with the highest individual Pearson correlation with target (+0.46) found in the heatmap.
- oldpeak (ST Depression): Appears as a top feature in all models, which is expected given the clear class separation observed in the bivariate histograms.
- thalach (Maximum Heart Rate): Also consistently ranked in the top features, confirming the −0.42 correlation with target found in Part 2.

The consistency of these top features across very different model architectures (ensemble bagging vs. boosting) strongly suggests that thal, cp, ca, oldpeak, and thalach are the most genuinely informative predictors in this dataset for distinguishing heart disease patients.

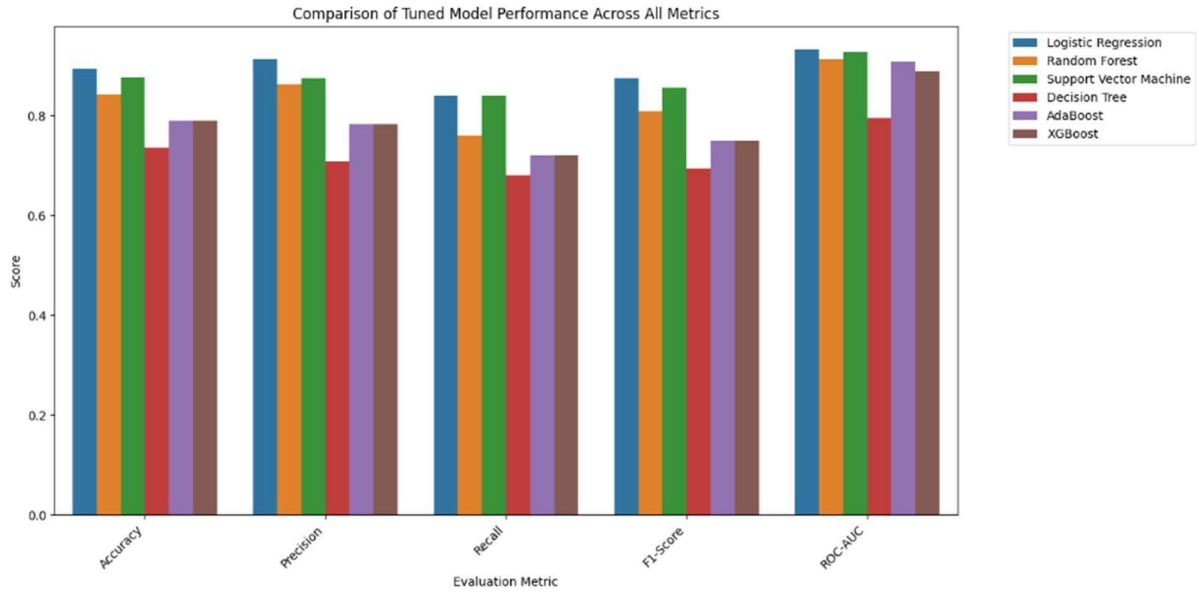
5.2 ROC Curve Comparison



The ROC (Receiver Operating Characteristic) curve plots the True Positive Rate (Recall/Sensitivity) against the False Positive Rate at every possible classification threshold. A larger area under the curve (AUC) indicates a better-performing model at all thresholds, not just the default 0.5 cutoff (Fawcett, 2006). The curves confirm the metric table findings: Logistic Regression (AUC 0.93) and SVM (AUC 0.93) occupy the top-left region of the plot closest to the

ideal corner while the Decision Tree curve hugs the diagonal most closely, reflecting its limited discriminating ability.

5.3 Overall Comparison of Model Performance



99 Comparison of Model Performance

After training, tuning, and evaluating all six traditional ML models, the following conclusions can be drawn:

Logistic Regression is the best-performing traditional model overall, achieving the highest accuracy (0.8947), highest Precision (0.9130), and highest F1-Score (0.8750) after tuning. This is a particularly significant result because Logistic Regression is also the simplest and most interpretable model its coefficients can be directly examined to understand how each feature influences predictions, which is a major advantage in a clinical decision-support context.

SVM is the strongest alternative, closely matching Logistic Regression on Recall and F1-Score. For clinical screening purposes where Recall is the priority, SVM's performance is effectively equal to Logistic Regression's on this dataset.

The ensemble models (Random Forest, AdaBoost, XGBoost) all achieve ROC-AUC values above 0.90 after tuning, confirming strong ranking ability, but their threshold-based accuracy metrics are lower than Logistic Regression and SVM. On larger datasets, ensemble methods typically

outperform linear models, but on this 228-sample training set, the simpler models generalise more reliably.

The Decision Tree, despite improvement through tuning, remains the weakest model and the only one flagged for possible overfitting. It should not be used as a standalone classifier for this task.

6. Overall Results, Insights and Challenges

6.1 Overall Results

The experimental results indicate considerable disparities in performance across the implemented models.

Logistic Regression emerged as the best-performing model following hyperparameter adjustment, achieving the maximum accuracy (0.8947), precision (0.9130), and F1-score (0.8750). It also maintained a robust ROC-AUC score of roughly 0.93, showing excellent classification and discriminating capacity.

Support Vector Machine (SVM) followed closely, with balanced performance across all measures and high generalisation ability, notably in terms of recall, which is crucial in medical diagnosis.

Ensemble models such as Random Forest, AdaBoost, and XGBoost displayed significant ROCAUC values (above 0.90), indicating good ranking ability. However, their total accuracy and F1scores were slightly lower than Logistic Regression and SVM, suggesting that they did not generalise as successfully on this relatively small dataset.

The Decision Tree model demonstrated the worst performance across all metrics, even after adjustment. Although improvements were found, the model remained prone to overfitting and demonstrated poor generalisation compared to other models.

Overall, hyperparameter adjustment increased the performance of most models, particularly Logistic Regression, SVM, and Decision Tree, whereas ensemble models showed mixed results due to dataset size limits.

6.2 Key Insights

Several noteworthy insights were observed from this study:

- One of the most notable discoveries is that simpler models, particularly Logistic Regression, outperformed more sophisticated ensemble approaches. This means that the

dataset reveals reasonably linear or moderately complicated interactions that can be adequately recorded without requiring highly sophisticated models.

- Another crucial lesson is the necessity of feature relevance. Features such as chest pain type (cp), ST depression (oldpeak), number of vessels (ca), maximal heart rate (thalach), and thalassemia (thal) consistently emerged as the most influential predictors across all models. This consistency reinforces their clinical value in identifying cardiac disease.
- The analysis also underlines the importance of evaluation indicators beyond accuracy. While accuracy gives a generic measure, measures such as recall and ROC-AUC are particularly relevant in medical applications. High recall guarantees that most patients with heart disease are appropriately diagnosed, lowering the chance of missed diagnosis.
- Additionally, the ROC curve research indicated that Logistic Regression and SVM provide superior class separation, keeping high true positive rates while limiting false positives well.
- Another insight is related to model behaviour on tiny datasets. Complex models such as XGBoost and AdaBoost, which normally perform well on huge datasets, did not achieve superior outcomes here. This shows that simpler models may generalise better when data is restricted.

6.3 Challenges and Limitations

Despite the successful execution and analysis, some problems and limits were faced in this study. One key problem was the relatively modest dataset size (303 samples). This reduces the ability of complicated models to learn robust patterns and increases the risk of overfitting. It also introduces unpredictability in performance evaluation, where findings may vary depending on the train-test split.

Another drawback is the absence of missing values in the dataset due to earlier preparation. While this streamlines the research, it does not reflect real-world clinical data, which often contains incomplete or noisy information.

The use of outlier removal, while improving model stability, may also result in the loss of potentially relevant extreme cases. In medical datasets, excessive results might occasionally signify crucial circumstances rather than errors.

Feature encoding increases the dimensionality of the dataset, which may impair model performance, particularly for algorithms sensitive to high-dimensional spaces.

Additionally, the analysis is limited to a single dataset. The findings may not generalise to other populations or datasets with different characteristics. External validation using independent datasets would be necessary to confirm the robustness of the models.

Another constraint is related to interpretability. While Logistic Regression delivers interpretable results, ensemble models and more complex techniques are less transparent, making them tougher to apply in clinical decision-making situations.

7. Conclusion

This study successfully demonstrated the application of machine learning algorithms for heart disease prediction utilizing an organized and systematic approach. From data preprocessing and exploratory analysis to model training, tuning, and evaluation, each stage contributed to constructing a viable predictive framework.

The findings clearly suggest that Logistic Regression is the most effective model for this dataset, achieving the best overall performance across multiple evaluation metrics. Support Vector Machine also demonstrated to be a strong alternative, particularly in maintaining balanced classification performance.

Although ensemble models displayed great discrimination capacity, they did not beat simpler models in this scenario, underlining the necessity of picking models based on dataset properties rather than complexity alone.

The study also underlined the need of effective preprocessing, feature analysis, and evaluation procedures in constructing reliable machine learning models, especially in sensitive areas such as healthcare.

In conclusion, our effort illustrates that machine learning can be a strong tool for helping heart disease diagnosis, but rigorous model design, evaluation, and comprehension of data properties are required to get trustworthy and meaningful findings.

References

Aggarwal, C. C., 2017. Outlier Analysis. 2nd Edition ed. s.l.:Springer.

Breiman, L., 2001. Random Forests. Machine Learning, 45(1).

Chen, T. & Guestrin, C., 2016. XGBoost: A scalable tree boosting system. s.l., ACM.

Cortes, C. & Vapnik, V., 1995. Support-vector networks. Machine Learning, Volume 20.

Detrano, R. J. A. S. W. P. M. S. J. J. S. S. G. K. L. S. &. F. V., 1989. International application of a new probability algorithm for the diagnosis of coronary artery disease. International application of a new probability algorithm for the diagnosis of coronary artery disease, Issue American Journal of Cardiology.

Fawcett, T., 2006. An introduction to ROC analysis. Pattern Recognition Letters, 27(8).

Freund, Y. & Schapire, R., 1997. A decision-theoretic generalization of on-line learning and an application to boosting. Journal of Computer and System Sciences, 55(1).

García, S. L. J. &. H. F., 2015. Data Preprocessing in Data Mining. s.l.:Springer.

Géron, A., 2022. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. 3rd Edition ed. s.l.:s.n.

Hosmer, D. & Lemeshow, S., 2000. Applied Logistic Regression. s.l.:Wiley.

Hunter, J. D., 2007. Matplotlib: A 2D Graphics Environment. Matplotlib: A 2D Graphics Environment.

Kohavi, R., 1995. A study of cross-validation and bootstrap for accuracy estimation and model selection. s.l., s.n.

McKinney, W., 2010. Data Structures for Statistical Computing in Python. s.l., Proceedings of the 9th Python in Science Conference (SciPy).

Pedregosa, F. et al., 2011. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, Volume 12.

Pedregosa, F. V. G. G. A. M. V. T. B. G. O. B. M. P. P. W. R. D. V. V. J. P. A. C. D. B. M. P. M. &. D. É., 2011. Scikit-learn: Machine Learning in Python.

Quinlan, J., 1986. Induction of decision trees. Machine Learning, 1(1).

Tukey, J. W., 1977. Exploratory Data Analysis. s.l.:s.n.